

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ASSIGNMENT 04 MÔN HỌC PHÁT TRIỂN
CÁC HỆ THỐNG THÔNG MINH

Giảng viên hướng dẫn: PGS.TS. Trần Đình Quế

Họ và tên: Nguyễn Chí Hùng

Mã sinh viên: B23DCCN359

Lớp: D23CNPM04

Nhóm lớp: 05

MỤC LỤC

PHẦN I: CƠ SỞ LÝ THUYẾT	6
CHƯƠNG 1: NỀN TẢNG TOÁN HỌC CỦA DEEP LEARNING	6
1.1. Hàm số – Khởi điểm toán học	6
1.2. Nơ-ron tuyến tính (Linear Neuron)	6
1.3. Tại sao cần hàm kích hoạt (Activation Functions)?	7
1.4. Các hàm kích hoạt quan trọng	8
1.5. Một lớp (Layer) là một hàm số	9
1.6. Deep Learning = Tổ hợp hàm (Function Composition)	9
CHƯƠNG 2: TỪ MẠNG KẾT NỐI ĐẦY ĐỦ ĐẾN XỬ LÝ ẢNH	10
2.1. Biểu diễn ảnh	10
2.2. Hạn chế của lớp kết nối đầy đủ	10
CHƯƠNG 3: TÍCH CHẬP (CONVOLUTION)	11
3.1. Ý tưởng cơ bản	11
3.2. Tại sao tích chập hữu ích?	11
3.3. Kích thước đầu ra của tích chập	12
3.4. Tích chập trong PyTorch	12
3.5. Kích hoạt sau tích chập	13
3.6. Pooling (Gộp)	14
3.7. Tích chập với stride	14
3.8. Chuẩn hóa theo lô (Batch Normalization)	15
CHƯƠNG 4: MỘT CNN NHỎ HOÀN CHỈNH	15
4.1. Kiến trúc cơ bản	15
4.2. Bộ phân loại và Softmax	17
4.3. Huấn luyện CNN	18
CHƯƠNG 5: HỆ THỐNG PHÂN CẤP BIỂU DIỄN CỦA CNN	19
5.1. Mô hình khái niệm	19
CHƯƠNG 6: SỰ TIẾN HÓA CỦA CÁC KIẾN TRÚC CNN	19
6.1. LeNet	20
6.2. AlexNet	20
6.3. VGG	20

6.4. Inception.....	21
6.5. Residual Networks (ResNet).....	21
6.5.2. Khối residual trong PyTorch.....	21
6.6. DenseNet.....	22
6.7. U-Net.....	23
6.8. MobileNet.....	23
6.9. EfficientNet	24
CHƯƠNG 7: ATTENTION (CƠ CHẾ CHÚ Ý)	24
7.1. Giới thiệu.....	24
7.2. Channel Attention: SE Block	25
7.3. Spatial Attention.....	26
7.4. CBAM.....	26
7.5. Tại sao kết hợp CNN + Attention?	27
CHƯƠNG 8: VISION TRANSFORMER (ViT)	27
8.1. Giới thiệu.....	27
8.2. Self-Attention	28
8.3. So sánh CNN và Transformer	28
CHƯƠNG 9: CÁI NHÌN THỐNG NHẤT VỀ SỰ TIẾN HÓA CỦA CNN	29
9.1. Tổng quan	29
9.2. Diễn giải Vấn đề – Giải pháp	30
CHƯƠNG 10: ĐỌC MÃ CNN MỘT CÁCH HỆ THỐNG.....	30
CHƯƠNG 11: MỘT CNN HOÀN CHỈNH	31
11.1. Thí nghiệm: Kiểm tra Shape	33
11.2. Đếm tham số.....	33
CHƯƠNG 12: TỪ NUMPY ĐẾN PYTORCH VÀ TENSORFLOW.....	34
12.1. Tích chập 1D – Định nghĩa toán học	34
12.2. Tích chập + ReLU	34
12.3. Từ Scratch đến PyTorch.....	35
12.4. TensorFlow/Keras	35
12.5. Ba cách triển khai.....	36

12.6. Kiến trúc theo tác vụ.....	36
PHẦN II: THỰC HÀNH	37
Chương 1: MNIST	37
1. TỔNG QUAN BÀI TOÁN VÀ KIẾN TRÚC MÔ HÌNH.....	37
2. PHẦN I: TRIỂN KHAI CNN BẰNG NUMPY THUẦN (VECTORIZED FROM SCRATCH).....	37
3. PHẦN II: TRIỂN KHAI CNN BẰNG PYTORCH	51
4. PHẦN III: TRIỂN KHAI CNN BẰNG TENSORFLOW / KERAS	59
5. PHẦN IV: BẢNG SO SÁNH TỔNG HỢP VÀ ĐÁNH GIÁ ĐA CHIỀU TRÊN MNIST.....	66
Chương 2: Fashion MNIST	67
1. TỔNG QUAN VÀ ĐẶC THÙ BÀI TOÁN FASHION-MNIST	67
2. PHẦN I: TRIỂN KHAI CNN BẰNG NUMPY THUẦN (VECTORIZED FROM SCRATCH).....	68
3. PHẦN II: TRIỂN KHAI CNN BẰNG PYTORCH	82
4. PHẦN III: TRIỂN KHAI CNN BẰNG TENSORFLOW / KERAS	90
5. PHẦN IV: BẢNG SO SÁNH TỔNG HỢP VÀ ĐÁNH GIÁ ĐA CHIỀU (NUMPY vs. PYTORCH vs. TENSORFLOW)	98

PHẦN I: CƠ SỞ LÝ THUYẾT

CHƯƠNG 1: NỀN TẢNG TOÁN HỌC CỦA DEEP LEARNING

1.1. Hàm số – Khởi điểm toán học

Trước khi nghiên cứu mạng nơ-ron, ta cần hiểu khái niệm hàm số. Một hàm số ánh xạ từ không gian đầu vào đến không gian đầu ra:

$$f: \mathbb{R}^d \rightarrow \mathbb{R}^m$$

Ví dụ đơn giản:

$$f(x) = 2x + 1$$

Nếu $x = 3$ thì $f(3) = 7$.

Mạng nơ-ron cũng là một hàm số. Điểm khác biệt căn bản là: **hàm số này không được chỉ định thủ công mà được học từ dữ liệu**. Với hàm có tham số:

$$f_{\theta}(x)$$

Vector tham số θ được học trong quá trình huấn luyện. Bài toán học trở thành:

Tìm θ sao cho $f_{\theta}(x)$ tạo ra đầu ra hữu ích.

1.2. Nơ-ron tuyến tính (Linear Neuron)

Nơ-ron đơn giản nhất thực hiện phép tính:

$$z = \sum_{i=1}^d w_i x_i + b$$

Dưới dạng vector:

$$z = w^T x + b$$

Trong đó:

- x : đầu vào (input)
- w : vector trọng số (weight vector)
- b : độ lệch (bias)

- z : đầu ra trước khi qua hàm kích hoạt

Ví dụ số:

Giả sử $w = [0.5, -, 12]$, $x = [2, 3, 1]$, $b = 0.5$:

$$z = 0.5(2) - 1(3) + 2(1) + 0.5 = 1 - 3 + 2 + 0.5 = 0.5$$

Neuron đã thực hiện hàm $f(x) = w^T x + b$.

Mã PyTorch tương ứng:

```
python
```

```
import torch
```

```
import torch.nn as nn
```

```
x = torch.tensor([[2.0, 3.0, 1.0]])
```

```
neuron = nn.Linear(3, 1)
```

```
with torch.no_grad():
```

```
    neuron.weight[:] = torch.tensor([[0.5, -1.0, 2.0]])
```

```
    neuron.bias[:] = torch.tensor([0.5])
```

```
y = neuron(x)
```

```
print(y)
```

1.3. Tại sao cần hàm kích hoạt (Activation Functions)?

Giả sử ta tổ hợp hai hàm tuyến tính:

$$f_1(x) = a_1x + b_1$$

$$f_2(x) = a_2x + b_2$$

Khi đó:

$$f_2(f_1(x)) = a_2(a_1x + b_1) + b_2 = (a_2a_1)x + (a_2b_1 + b_2)$$

Kết quả vẫn là hàm tuyến tính – affine. Do đó:

Tuyến tính • Tuyến tính = Tuyến tính

Một mạng sâu chỉ chứa các lớp tuyến tính sẽ không có sức mạnh biểu diễn như kỳ vọng của học sâu. Vì vậy, ta cần đưa vào **hàm phi tuyến**:

$$h = \sigma(z)$$

1.4. Các hàm kích hoạt quan trọng

1.4.1. ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

Ví dụ: $\text{ReLU}(-2) = 0$, $\text{ReLU}(3) = 3$.

```
python
```

```
import torch
```

```
import torch.nn.functional as F
```

```
x = torch.tensor([-2.0, -0.5, 0.0, 1.0, 3.0])
```

```
y = F.relu(x)
```

```
print(y)
```

1.4.2. Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Phạm vi: $0 < \sigma(x) < 1$. Hữu ích cho xác suất nhị phân, nhưng các lớp ẩn hiện đại thường ưu tiên ReLU.

1.4.3. Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Phạm vi: $-1 < \tanh(x) < 1$.

1.4.4. GELU và SiLU

Các kiến trúc hiện đại thường sử dụng:

$$\text{GELU}(x) \text{ hoặc } \text{SiLU}(x) = x\sigma(x)$$

Việc chọn hàm kích hoạt phụ thuộc vào kiến trúc cụ thể.

1.5. Một lớp (Layer) là một hàm số

Một lớp nơ-ron nên được hiểu như một hàm số. Ví dụ:

$$f(x) = \text{ReLU}(Wx + b)$$

Trong PyTorch:

python

```
layer = nn.Sequential(  
    nn.Linear(10, 20),  
    nn.ReLU()  
)
```

Toán học:

$$f(x) = \text{ReLU}(Wx + b)$$

Do đó: **Layer** $\leftrightarrow$ **Function**.

1.6. Deep Learning = Tổ hợp hàm (Function Composition)

Xét:

$$f_1(x) = W_1x + b_1$$

$$f_2(h) = \text{ReLU}(h)$$

$$f_3(h) = W_2h + b_2$$

Mạng là:

$$F(x) = f_3 \circ f_2 \circ f_1(x)$$

Cụ thể:

$$F(x) = W_2 \text{ReLU}(W_1x + b_1) + b_2$$

Đây là ý tưởng toán học nền tảng của học sâu.

CHƯƠNG 2: TỪ MẠNG KẾT NỐI ĐẦY ĐỦ ĐẾN XỬ LÝ ẢNH

2.1. Biểu diễn ảnh

Một ảnh màu có thể được biểu diễn như:

$$X \in \mathbb{R}^{C \times H \times W}$$

Trong đó:

- C : số kênh (channels)
- H : chiều cao (height)
- W : chiều rộng (width)

Với ảnh RGB: $C = 3$.

Với ảnh kích thước 224×224 :

$$X \in \mathbb{R}^{3 \times 224 \times 224}$$

Khi làm phẳng (flatten):

$$3 \cdot 224 \cdot 224 = 150,528 \text{ số}$$

Một lớp kết nối đầy đủ từ vector này đến 1000 đầu ra cần khoảng:

$$150,528 \times 1000 \approx 150 \text{ triệu tham số}$$

Điều này thúc đẩy nhu cầu về một phép toán phù hợp hơn: **Tích chập (Convolution)**.

2.2. Hạn chế của lớp kết nối đầy đủ

Lớp kết nối đầy đủ kết nối mọi đặc trưng đầu vào với mọi nơ-ron:

$$H = \sigma(XW + b)$$

Điều này mạnh mẽ nhưng **bỏ qua cấu trúc cục bộ**. Với ảnh, pixel tại (i, j) có liên quan tự nhiên đến các pixel lân cận:

$$(i, -1j), (i, +1j), (i, j - 1), (i, j + 1)$$

Với văn bản, từ có liên quan đến các từ gần đó.

CHƯƠNG 3: TÍCH CHẬP (CONVOLUTION)

3.1. Ý tưởng cơ bản

Một kernel tích chập là một ma trận nhỏ. Tại mỗi vị trí ảnh, kernel tương tác với một vùng ảnh cục bộ. Với một kernel:

$$Y(i, j) = \sum_{u=1}^k \sum_{v=1}^k K(u, v) X(i + u, j + v)$$

Đầu ra Y được gọi là **feature map** (bản đồ đặc trưng).

Ví dụ số đơn giản:

Giả sử:

$$X = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}, K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Đầu ra đầu tiên:

$$Y(1, 1) = 1(1) + 0(2) + 0(4) - 1(5) = -4$$

Tích chập là một **tổng có trọng số cục bộ**.

3.2. Tại sao tích chập hữu ích?

Ba ý tưởng nền tảng:

1. **Kết nối cục bộ (Local connectivity)**
2. **Chia sẻ trọng số (Weight sharing)**
3. **Tính toán có cấu trúc không gian (Spatially structured computation)**

3.2.1. Kết nối cục bộ

Một nơ-ron tích chập chỉ nhìn thấy một vùng lân cận nhỏ. Với kernel 3×3 :

$$\boxed{\text{receptive field} = 3 \times 3}$$

tại lớp đó.

3.2.2. Chia sẻ trọng số

Cùng một kernel được sử dụng tại nhiều vị trí. Do đó, mạng không cần học một bộ phát hiện riêng cho từng vị trí pixel. Điều này **giảm mạnh số lượng tham số**.

3.3. Kích thước đầu ra của tích chập

Với một chiều không gian:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Trong đó:

- H : kích thước đầu vào
- K : kích thước kernel
- P : padding
- S : stride

Với ảnh vuông:

$$W_{\text{out}} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

Ví dụ:

Cho $H = W = 32, K = 3, P = 1, S = 1$:

$$H_{\text{out}} = \frac{32 + 2 - 3}{1} + 1 = 32$$

Vậy tích chập 3×3 với padding 1 và stride 1 **giữ nguyên độ phân giải không gian**.

3.4. Tích chập trong PyTorch

python

```
import torch
```

```
import torch.nn as nn
```

```
conv = nn.Conv2d(
    in_channels=3,
    out_channels=16,
    kernel_size=3,
    stride=1,
    padding=1
)
x = torch.randn(8, 3, 32, 32)
y = conv(x)
print(x.shape) # torch.Size([8, 3, 32, 32])
print(y.shape) # torch.Size([8, 16, 32, 32])
```

Các tensor shape:

$$(8, 3, 32, 32) \rightarrow (8, 16, 32, 32)$$

Chiều đầu tiên là batch size. Tích chập đã học 16 kernel, mỗi kernel hoạt động trên 3 kênh đầu vào. Số tham số:

$$16 \times 3 \times 3 \times 3 + 16 = 448$$

3.5. Kích hoạt sau tích chập

Một khối CNN điển hình:

$$X \rightarrow \text{Conv} \rightarrow \text{ReLU}$$

Toán học:

$$Y = \text{ReLU}(K * X + b)$$

Trong PyTorch:

python

```
block = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, padding=1),
```

```
nn.ReLU()  
)
```

Đây lại là **tổ hợp hàm**.

3.6. Pooling (Gộp)

Max pooling được định nghĩa bởi:

$$Y(i, j) = \max_{(u, v) \in \mathcal{N}(i, j)} X(i, +uj + v)$$

Lựa chọn phổ biến: pooling 2×2 với stride 2. Do đó:

$$32 \times 32 \rightarrow 16 \times 16$$

```
python
```

```
pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
x = torch.randn(8, 16, 32, 32)
```

```
y = pool(x)
```

```
print(y.shape) # torch.Size([8, 16, 16, 16])
```

Pooling chủ yếu cung cấp **giảm kích thước không gian** và **tăng receptive field hiệu quả** của các lớp sau.

3.7. Tích chập với stride

Thay vì:

Conv $\rightarrow$ Pool

mạng có thể dùng:

Conv(stride = 2)

Ví dụ:

```
python
```

```
downsample = nn.Conv2d(  
    16, 32,  
    kernel_size=3,  
    stride=2,  
    padding=1,  
    bias=False)
```

```
stride=2,  
padding=1  
)
```

Điều này thay đổi:

$$32 \times 32 \rightarrow 16 \times 16$$

Các kiến trúc hiện đại thường dùng **tích chập stride** để giảm mẫu có học.

3.8. Chuẩn hóa theo lô (Batch Normalization)

Batch normalization biến đổi các activation bằng cách chuẩn hóa phụ thuộc batch, sau đó scale và shift có học:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Sau đó:

$$y = \gamma \hat{x} + \beta$$

python

```
block = nn.Sequential(  
    nn.Conv2d(3, 32, 3, padding=1),  
    nn.BatchNorm2d(32),  
    nn.ReLU()  
)
```

Vị trí chính xác của chuẩn hóa thay đổi giữa các kiến trúc.

CHƯƠNG 4: MỘT CNN NHỎ HOÀN CHỈNH

4.1. Kiến trúc cơ bản

Một bộ phân loại ảnh cơ bản:

$X \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}$
 $\rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool}$
 $\rightarrow \text{Flatten} \rightarrow \text{Linear}$

Triển khai PyTorch:

```
python
```

```
import torch
```

```
import torch.nn as nn
```

```
class SmallCNN(nn.Module):
```

```
    def __init__(self, num_classes=10):
```

```
        super().__init__()
```

```
        self.features = nn.Sequential(
```

```
            nn.Conv2d(3, 32, 3, padding=1),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(2),
```

```
            nn.Conv2d(32, 64, 3, padding=1),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(2)
```

```
        )
```

```
        self.classifier = nn.Sequential(
```

```
            nn.Flatten(),
```

```
            nn.Linear(64 * 8 * 8, 128),
```

```
            nn.ReLU(),
```

```
            nn.Linear(128, num_classes)
```

```
        )
```

```
    def forward(self, x):
```

```
        x = self.features(x)
```

```
        x = self.classifier(x)
```

```
        return x
```

```

model = SmallCNN(num_classes=10)
x = torch.randn(4, 3, 32, 32)
logits = model(x)
print(logits.shape) # torch.Size([4, 10])

```

Tiến trình shape:

```

3 × 32 × 32
↓ Conv
32 × 32 × 32
↓ Pool
32 × 16 × 16
↓ Conv
64 × 16 × 16
↓ Pool
64 × 8 × 8

```

Sau đó:

$$64 \cdot 8 \cdot 8 = 4096$$

đặc trưng được làm phẳng.

4.2. Bộ phân loại và Softmax

Bộ phân loại tạo ra **logits**:

$$z = [z_1, \dots, z_c]$$

Softmax chuyển logits thành xác suất:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^c e^{z_j}}$$

Do đó:

$$\sum_i p_i = 1$$

Trong PyTorch, khi dùng cross entropy loss, thường nên truyền raw logits:

python

```
criterion = nn.CrossEntropyLoss()
```

```
logits = model(x)
```

```
loss = criterion(logits, labels)
```

Không tự động áp dụng softmax trước CrossEntropyLoss trong cấu hình phân loại thông thường.

4.3. Huấn luyện CNN

Huấn luyện gồm bốn thao tác chính:

forward → loss → backward → update

python

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
criterion = nn.CrossEntropyLoss()
```

```
model.train()
```

```
for images, labels in train_loader:
```

```
    optimizer.zero_grad()
```

```
    logits = model(images)
```

```
    loss = criterion(logits, labels)
```

```
    loss.backward()
```

```
    optimizer.step()
```

Toán học:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

Huấn luyện là một **bài toán tối ưu hóa**.

CHƯƠNG 5: HỆ THỐNG PHÂN CẤP BIỂU DIỄN CỦA CNN

5.1. Mô hình khái niệm

Một cách diễn giải khái niệm hữu ích:

pixels → edges → textures → parts → objects → class

Điều này **không nên** được hiểu như một đảm bảo chính xác cho mọi CNN. Thay vào đó, nó là một mô hình khái niệm hữu ích để hiểu việc học đặc trưng phân cấp.

CHƯƠNG 6: SỰ TIẾN HÓA CỦA CÁC KIẾN TRÚC CNN

Sự tiến hóa có thể được hiểu như một chuỗi các **vấn đề và giải pháp**.

Mô hình/họ	Vấn đề chính	Ý tưởng chính
LeNet	Cấu trúc ảnh không gian	Tích chập + pooling
AlexNet	Học sâu quy mô lớn	ReLU, GPU, dropout
VGG	Cần biểu diễn mạnh hơn	Tích chập 3×3 lặp lại
Inception	Nhiều tỉ lệ đối tượng	Nhánh đa tỉ lệ
ResNet	Tối ưu hóa rất sâu	Kết nối tắt (skip connection)
DenseNet	Lan truyền thông tin	Nối đặc trưng (concatenation)
U-Net	Dự đoán dày đặc	Encoder-decoder + skip
MobileNet	Chi phí tính toán	Tích chập tách biệt depthwise
EfficientNet	Mở rộng CNN	Compound scaling

Mô hình/họ	Vấn đề chính	Ý tưởng chính
SE-Net	Tầm quan trọng kênh	Channel attention
CBAM	Kênh và không gian	Channel + spatial attention

6.1. LeNet

LeNet thiết lập mẫu CNN cơ bản:

Conv → Activation → Pooling → Conv → Pooling → FC

Tầm quan trọng của nó là **khái niệm**: học đặc trưng không gian trực tiếp từ ảnh.

6.2. AlexNet

AlexNet chứng minh rằng một CNN sâu hơn được huấn luyện ở quy mô lớn có thể cải thiện đáng kể phân loại ảnh.

Các ý tưởng quan trọng:

- ReLU activation
- Tăng tốc GPU
- Dropout
- Tăng cường dữ liệu (data augmentation)
- Biểu diễn tích chập sâu hơn

Bài học chính: **quy mô lớn + học sâu có thể tạo ra biểu diễn thị giác hiệu quả cao.**

6.3. VGG

VGG nhấn mạnh **sự đơn giản kiến trúc**. Thay vì kernel rất lớn, nó dùng tích chập 3×3 lặp lại.

Ví dụ, 7×7 có thể được thay thế khái niệm bằng một chuỗi tích chập nhỏ hơn:

$\text{Conv}_{3 \times 3} \rightarrow \text{ReLU} \rightarrow \text{Conv}_{3 \times 3} \rightarrow \text{ReLU}$

Bài học quan trọng: **độ sâu + phép toán cục bộ nhỏ.**

6.4. Inception

Một kích thước kernel duy nhất có thể không tối ưu cho mọi cấu trúc thị giác. Inception kết hợp nhiều phép toán:

$$1 \times 1, 3 \times 3, 5 \times 5, \text{ Pooling}$$

song song, sau đó nối lại.

Khái niệm:

$$Y = \text{Concat} \left(f_{1 \times 1}(X), f_{3 \times 3}(X), f_{5 \times 5}(X), f_{\text{pool}}(X) \right)$$

Vấn đề được giải quyết: **biểu diễn đa tỉ lệ**.

6.5. Residual Networks (ResNet)

6.5.1. Vấn đề suy giảm (Degradation Problem)

Tăng độ sâu không tự động đảm bảo tối ưu hóa tốt hơn.

Một khối residual thay đổi **mục tiêu học**. Thay vì học $H(X)$, khối học:

$$F(X) = H(X) - X$$

Sau đó:

$$Y = F(X) + X$$

Đường đồng nhất (identity path) cho phép thông tin và gradient truyền trực tiếp hơn.

6.5.2. Khối residual trong PyTorch

python

```
class ResidualBlock(nn.Module):
```

```
    def __init__(self, channels):
```

```
        super().__init__()
```

```
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
```

```
        self.bn1 = nn.BatchNorm2d(channels)
```

```
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
```

```
self.bn2 = nn.BatchNorm2d(channels)
```

```
self.relu = nn.ReLU()
```

```
def forward(self, x):
```

```
    identity = x
```

```
    out = self.conv1(x)
```

```
    out = self.bn1(out)
```

```
    out = self.relu(out)
```

```
    out = self.conv2(out)
```

```
    out = self.bn2(out)
```

```
    out = out + identity
```

```
    out = self.relu(out)
```

```
    return out
```

Dòng quan trọng:

python

```
out = out + identity
```

thực hiện:

$$Y = F(X) + X$$

6.6. DenseNet

DenseNet sử dụng **nối đặc trưng**:

$$X_l = H_l([X_0, X_1, \dots, X_{l-1}])$$

Dấu ngoặc vuông biểu thị phép nối (concatenation).

Ý tưởng chính: **tái sử dụng đặc trưng**.

Mạng residual dùng phép cộng:

$$F(X) + X$$

Mạng dense dùng phép nối:

$$[X, F(X)]$$

Đây là một **khác biệt kiến trúc quan trọng**.

6.7. U-Net

U-Net giải quyết **dự đoán dày đặc** (dense prediction).

Phân loại hồi:

$$\text{Image} \rightarrow \text{Class}$$

Phân đoạn hồi:

$$\text{Image} \rightarrow \text{label cho mỗi pixel}$$

U-Net dùng kiến trúc encoder-decoder:

$$\text{Image} \rightarrow \text{Encoder} \rightarrow \text{Bottleneck} \rightarrow \text{Decoder} \rightarrow \text{Segmentation}$$

Skip connection truyền thông tin không gian độ phân giải cao từ encoder sang decoder.

Ý tưởng quan trọng: **kết hợp thông tin ngữ nghĩa với chi tiết không gian**.

6.8. MobileNet

Tích chập tiêu chuẩn thực hiện **trộn không gian và kênh** cùng lúc. Tích chập tách biệt depthwise tách hai phép toán.

Đầu tiên, depthwise convolution:

$$X \rightarrow \text{lọc không gian}$$

Sau đó, pointwise convolution:

$$1 \times 1$$

để trộn kênh.

Khái niệm:

Depthwise Conv + Pointwise Conv

Điều này **giảm đáng kể tính toán**.

python

```
depthwise = nn.Conv2d(32, 32, kernel_size=3, padding=1, groups=32)
```

```
pointwise = nn.Conv2d(32, 64, kernel_size=1)
```

Điều kiện groups=32 làm cho tích chập đầu tiên trở thành depthwise cho 32 kênh đầu vào.

6.9. EfficientNet

Hiệu năng CNN phụ thuộc vào:

- độ sâu (depth)
- độ rộng (width)
- độ phân giải đầu vào (resolution)

Tăng ngẫu nhiên chỉ một yếu tố có thể không hiệu quả. EfficientNet giới thiệu **chiến lược compound scaling có nguyên tắc**.

Khái niệm:

depth ↑, width ↑, resolution ↑

được scale cùng nhau theo một quy tắc có kiểm soát.

Bài học quan trọng: **mở rộng kiến trúc tự nó là một bài toán tối ưu hóa**.

CHƯƠNG 7: ATTENTION (CƠ CHẾ CHÚ Ý)

7.1. Giới thiệu

CNN áp dụng bộ lọc cục bộ. Nhưng không phải mọi đặc trưng đều quan trọng như nhau.

Attention giới thiệu cơ chế **gán tầm quan trọng**.

Một phép tổng hợp có trọng số đơn giản:

$$Y_i = \sum_j a_{ij} X_j$$

Trong đó:

$$a_{ij} \geq 0, \sum_j a_{ij} = 1$$

Các trọng số a_{ij} xác định mức độ tương tác giữa các đặc trưng.

Do đó:

Attention = tương tác chọn lọc có học

7.2. Channel Attention: SE Block

Giả sử:

$$X \in \mathbb{R}^{C \times H \times W}$$

Cơ chế channel attention tạo ra:

$$s \in \mathbb{R}^C$$

Sau đó:

$$Y_c = s_c X_c$$

Triển khai SE block đơn giản:

python

```
class SEBlock(nn.Module):  
    def __init__(self, channels, reduction=4):  
        super().__init__()  
        hidden = max(channels // reduction, 1)  
        self.pool = nn.AdaptiveAvgPool2d(1)  
        self.fc = nn.Sequential(  
            nn.Linear(channels, hidden),  
            nn.ReLU(),  
            nn.Linear(hidden, channels),  
            nn.Sigmoid()
```

)

```
def forward(self, x):  
    b, c, _, _ = x.shape  
    s = self.pool(x).view(b, c)  
    s = self.fc(s).view(b, c, 1, 1)  
    return x * s
```

Phép toán thiết yếu:

$$Y = X \odot s$$

Điều này trả lời câu hỏi: **Kênh nào quan trọng?**

7.3. Spatial Attention

Spatial attention hỏi: **Thông tin quan trọng ở đâu?**

Một cơ chế spatial attention đơn giản có thể tạo ra:

$$A \in \mathbb{R}^{1 \times H \times W}$$

và tính:

$$Y = A \odot X$$

Do đó:

- Channel attention hỏi: **What?**

Spatial attention hỏi: Where?

7.4. CBAM

CBAM kết hợp:

Channel Attention $\rightarrow$ Spatial Attention

Biến đổi đặc trưng có thể được viết khái niệm:

$$X \rightarrow M_c(X) \odot X \rightarrow M_s(X) \odot X$$

Đây là một ví dụ về việc tiến hóa CNN bằng cách **thêm một hàm mới vào tổ hợp**.

7.5. Tại sao kết hợp CNN + Attention?

Tích chập với kernel 3×3 chủ yếu nắm bắt **quan hệ cục bộ**. Với ảnh:

$$\text{pixel}_{i,j}$$

tương tác trực tiếp với các pixel gần đó.

Tuy nhiên, một số tác vụ yêu cầu quan hệ giữa các vùng xa. Ví dụ:

$$\text{head} \leftrightarrow \text{body}$$

có thể liên quan đến các vùng không gian tách biệt.

Attention có thể cung cấp **tương tác tầm xa trực tiếp hơn**.

Do đó:

- **CNN**: inductive bias cục bộ
- **Attention**: tương tác linh hoạt/toàn cục

CHƯƠNG 8: VISION TRANSFORMER (ViT)

8.1. Giới thiệu

Vision Transformer thay đổi **chiến lược biểu diễn**.

Một ảnh được chia thành các patch. Với ảnh kích thước:

$$224 \times 224$$

và patch size:

$$16 \times 16$$

số lượng patch là:

$$\frac{224}{16} \times \frac{224}{16} = 14 \times 14 = 196$$

Mỗi patch được làm phẳng và chiếu vào một embedding. Do đó:

Image $\rightarrow$ Patches $\rightarrow$ Tokens

Các token được xử lý bằng **self-attention**.

Pipeline cấp cao:

Image $\rightarrow$ Patch Embedding $\rightarrow$ Transformer $\rightarrow$ Classifier

8.2. Self-Attention

Cho các biểu diễn token, attention sử dụng queries, keys và values:

$$Q = XW_Q, K = XW_K, V = XW_V$$

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Điều này **cơ bản khác** với tích chập cục bộ cố định.

8.3. So sánh CNN và Transformer

	CNN	Transformer
Phép toán cơ bản	Convolution	Self-attention
Tương tác	Cục bộ	Có thể toàn cục
Inductive bias	Cục bộ không gian mạnh	Ít bias cục bộ hơn
Chia sẻ trọng số	Chia sẻ kernel	Chia sẻ projection
Biểu diễn	Feature maps	Token sequences

	CNN	Transformer
Điểm mạnh điển hình	Cấu trúc thị giác cục bộ	Quan hệ tầm xa

So sánh **không nên** được hiểu là "một cái luôn tốt hơn". Các hệ thống thị giác hiện đại thường **kết hợp cả hai ý tưởng**.

CHƯƠNG 9: CÁI NHÌN THỐNG NHẤT VỀ SỰ TIẾN HÓA CỦA CNN

9.1. Tổng quan

Sự tiến hóa có thể được tóm tắt:

text

Neuron $\rightarrow$ Linear function

$\rightarrow$ Activation

$\rightarrow$ Layer

$\rightarrow$ Function composition

$\rightarrow$ Convolution

$\rightarrow$ Deep CNN

$\rightarrow$ Multi-scale CNN

$\rightarrow$ Residual CNN

$\rightarrow$ Efficient CNN

$\rightarrow$ Attention CNN

$\rightarrow$ CNN-Transformer

$\rightarrow$ Vision Transformer

Mỗi bước có thể được diễn giải như **thêm một hàm giải quyết hạn chế của biểu diễn trước đó**.

9.2. Diễn giải Vấn đề – Giải pháp

Vấn đề	Phép toán	Mục đích
Chỉ có biến đổi tuyến tính	Activation	Phi tuyến
Quá nhiều tham số ảnh	Convolution	Cục bộ + chia sẻ
Kích thước không gian lớn	Pooling/stride	Giảm mẫu
Cần đặc trưng phân cấp	Deep stacking	Phân cấp biểu diễn
Tối ưu hóa rất sâu	Residual connection	Dòng thông tin/gradient dễ hơn
Cần tái sử dụng đặc trưng	Dense connection	Lan truyền thông tin
Kích thước đối tượng khác nhau	Multi-scale branches	Đa dạng tỉ lệ
Tính toán cao	Depthwise convolution	Hiệu quả
Tầm quan trọng đặc trưng không đều	Attention	Trọng số chọn lọc
Quan hệ tầm xa	Self-attention	Tương tác toàn cục
Dự đoán pixel dày đặc	Encoder-decoder + skip	Tái tạo không gian

CHƯƠNG 10: ĐỌC MÃ CNN MỘT CÁCH HỆ THỐNG

Khi đọc một mô hình PyTorch, sinh viên nên hỏi các câu hỏi sau:

1. **Shape đầu vào là gì?** Ví dụ: $B \times 3 \times 224 \times 224$.
2. **Hàm nào được áp dụng đầu tiên?** Ví dụ: `nn.Conv2d(3, 32, 3, padding=1)` nghĩa là 3 kênh $\rightarrow$ 32 kênh.
3. **Kích thước không gian có thay đổi không?** Kiểm tra K, P, S .
4. **Có phi tuyến không?** Tìm `nn.ReLU()`, `nn.GELU()`, `nn.SiLU()`.
5. **Có giảm mẫu không?** Tìm `MaxPool2d`, `AdaptiveAvgPool2d`, `stride=2`.
6. **Có shortcut không?** Tìm `out = out + identity` – thường chỉ ra cấu trúc residual.
7. **Có concatenation không?** Tìm `torch.cat(...)` – có thể chỉ ra dense connections hoặc kiến trúc đa nhánh.
8. **Có attention không?** Tìm các phép toán tạo trọng số và nhân với đặc trưng: $x = x * \text{attention}$ hoặc tương tác token-to-token.

CHƯƠNG 11: MỘT CNN HOÀN CHỈNH

Mô hình sau đây cố ý kết hợp các ý tưởng cơ bản mà không dùng kiến trúc hiện đại phức tạp.

```
python
```

```
import torch
```

```
import torch.nn as nn
```

```
class EducationalCNN(nn.Module):
```

```
    def __init__(self, num_classes=10):
```

```
        super().__init__()
```

```
        self.block1 = nn.Sequential(
```

```
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
```

```
            nn.BatchNorm2d(32),
```

```
            nn.ReLU(),
```

```
            nn.MaxPool2d(2)
```

```
        )
```

```
        self.block2 = nn.Sequential(
```

```

nn.Conv2d(32, 64, kernel_size=3, padding=1),
nn.BatchNorm2d(64),
nn.ReLU(),
nn.MaxPool2d(2)
)
self.block3 = nn.Sequential(
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
    nn.BatchNorm2d(128),
    nn.ReLU(),
    nn.MaxPool2d(2)
)
self.head = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(),
    nn.Linear(128, num_classes)
)

```

```

def forward(self, x):
    x = self.block1(x)
    x = self.block2(x)
    x = self.block3(x)
    x = self.head(x)
    return x

```

Với:

$$X \in \mathbb{R}^{B \times 3 \times 224 \times 224}$$

kích thước không gian xấp xỉ:

$$224 \rightarrow 112 \rightarrow 56 \rightarrow 28$$

Adaptive average pooling sau đó tạo ra:

$$128 \times 1 \times 1$$

Do đó bộ phân loại chỉ nhận **128 giá trị**.

Đây là ví dụ hữu ích vì sinh viên có thể **theo dõi từng phép toán**.

11.1. Thí nghiệm: Kiểm tra Shape

Một kỹ thuật giáo dục rất hữu ích là in shape của tensor.

python

```
model = EducationalCNN(num_classes=10)
```

```
x = torch.randn(2, 3, 224, 224)
```

```
print("Input:", x.shape)
```

```
x = model.block1(x)
```

```
print("After block1:", x.shape)
```

```
x = model.block2(x)
```

```
print("After block2:", x.shape)
```

```
x = model.block3(x)
```

```
print("After block3:", x.shape)
```

```
x = model.head(x)
```

```
print("Output:", x.shape)
```

Sinh viên nên **dự đoán shape đầu ra trước khi chạy chương trình**. Khả năng đó là một trong những bài kiểm tra tốt nhất để biết liệu họ đã hiểu CNN chưa.

11.2. Đếm tham số

Với một lớp tích chập:

$$N_{\text{params}} = C_{\text{out}}C_{\text{in}}K_hK_w + C_{\text{out}}$$

nếu dùng bias.

Với:

$$\text{Conv2d}(3, 32, 3)$$

ta có:

$$32 \cdot 3 \cdot 3 \cdot 3 + 32 = 896$$

Với một lớp tuyến tính:

$$N_{\text{params}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}}$$

Điều này quan trọng vì thiết kế kiến trúc cũng là một **bài toán về tài nguyên tính toán**.

CHƯƠNG 12: TỪ NUMPY ĐẾN PYTORCH VÀ TENSORFLOW

12.1. Tích chập 1D – Định nghĩa toán học

Với đầu vào x và kernel k kích thước m :

$$z_i = \sum_{j=0}^{m-1} k_j x_{i+j} + b$$

Kernel di chuyển qua đầu vào và tạo ra một giá trị tại mỗi vị trí.

Ví dụ:

$$x = [2, 1, 3, 4, 2]$$
$$k = [0.5, -, 10.5], b = 0$$

Kernel size là 3.

$$z_1 = (2)(0.5) + (1)(-1) + (3)(10.5) = 1 - 1 + 31.5 = 31.5$$

12.2. Tích chập + ReLU

Phép toán tích chập đơn thuần chưa phải là một lớp mạng nơ-ron hoàn chỉnh.

$$z = \text{Conv1D}(x; k; b)$$

Áp dụng hàm kích hoạt phi tuyến:

$$h = \text{ReLU}(z)$$

với:

$$\text{ReLU}(z) = \max(0, z)$$

Ví dụ:

$$z = [1, -, 0.5 - 1.5]$$

thì:

$$h = [1, 0, 0]$$

12.3. Từ Scratch đến PyTorch

Phép toán được triển khai thủ công:

$$\text{Conv1D}(X, W, b)$$

trở thành khái niệm:

python

`nn.Conv1d(in_channels, out_channels, kernel_size)`

Hàm kích hoạt:

python

`nn.ReLU()`

và pooling:

python

`nn.MaxPool1d(...)`

12.4. TensorFlow/Keras

Cùng phép toán khái niệm có thể được biểu diễn bằng TensorFlow/Keras:

python

`tf.keras.layers.Conv1D(`

```

filters=16,
kernel_size=3,
activation="relu"
)

```

Pooling:

python

```
tf.keras.layers.MaxPooling1D(...)
```

và lớp đầu ra cho phân loại nhị phân:

python

```
tf.keras.layers.Dense(1, activation="sigmoid")
```

12.5. Ba cách triển khai

NumPy	PyTorch	TensorFlow
Tích chập thủ công	Conv1d	Conv1D
ReLU thủ công	ReLU	ReLU
Gradient thủ công	Autograd	GradientTape
Cập nhật thủ công	Optimizer	Optimizer
Vòng lặp huấn luyện thủ công	Training loop	fit() / custom loop

Đây là một bài học kỹ thuật phần mềm quan trọng:

Cùng một mô hình toán học $\neq$ cùng một triển khai

12.6. Kiến trúc theo tác vụ

- **Phân loại nhị phân:** Output = Sigmoid
- **Phân loại đa lớp:** Output = Softmax
- **Hồi quy:** Output = Linear

Do đó:

Kiến trúc mạng nơ-ron = Dữ liệu + Tác vụ + Biểu diễn

PHẦN II: THỰC HÀNH

Source code: [Hunguyenobugs/Phat-trien-cac-he-thong-thong-minh](https://github.com/Hunguyenobugs/Phat-trien-cac-he-thong-thong-minh)

Chương 1: MNIST

1. TỔNG QUAN BÀI TOÁN VÀ KIẾN TRÚC MÔ HÌNH

Bộ dữ liệu **MNIST** gồm 70.000 ảnh chữ số viết tay kích thước 28×28 điểm ảnh thang độ xám (Grayscale), chia thành 60.000 ảnh huấn luyện và 10.000 ảnh kiểm thử, trải đều trên 10 lớp ($0 \rightarrow 9$). Kiến trúc CNN chuẩn được áp dụng đồng nhất qua các nền tảng:

2. PHẦN I: TRIỂN KHAI CNN BẰNG NUMPY THUẦN (VECTORIZED FROM SCRATCH)

2.1. Cell 1: Cấu hình Môi trường & Thiết lập Thư mục

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os, time, struct, pickle
from pathlib import Path
from sklearn.metrics import confusion_matrix, classification_report

np.random.seed(42)
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 80

DATA_DIR = Path('../data')
MODEL_DIR = Path('../models')
MODEL_DIR.mkdir(exist_ok=True)
print('Data dir :', DATA_DIR.resolve())
print('Model dir:', MODEL_DIR.resolve())
```

Output:

Data dir : D:\Đại học\Phát triển các hệ thống thông minh\Assignment
04\mnist\data

Model dir: D:\Đại học\Phát triển các hệ thống thông minh\Assignment
04\mnist\models

Diễn giải kỹ thuật:

- Thiết lập seed ngẫu nhiên cố định 42 nhằm đảm bảo khả năng tái lập (reproducibility) trong việc khởi tạo trọng số và xáo trộn dữ liệu.

- Quản lý đường dẫn bằng pathlib.Path giúp mã nguồn hoạt động tương thích trên mọi hệ điều hành (Windows, Linux, macOS). Phương thức mkdir(exist_ok=True) tự động khởi tạo thư mục lưu trữ trọng số mô hình nếu chưa có.

2.2. Cell 2: Đọc dữ liệu nhị phân IDX chuẩn của MNIST

Code:

```
def load_mnist_images(path):
    with open(path, 'rb') as f:
        magic, num, rows, cols = struct.unpack('>IIII', f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(num, rows, cols)
    return data

def load_mnist_labels(path):
    with open(path, 'rb') as f:
        magic, num = struct.unpack('>II', f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train_full = load_mnist_images(DATA_DIR / 'train-images.idx3-ubyte')
y_train_full = load_mnist_labels(DATA_DIR / 'train-labels.idx1-ubyte')
X_test = load_mnist_images(DATA_DIR / 't10k-images.idx3-ubyte')
y_test = load_mnist_labels(DATA_DIR / 't10k-labels.idx1-ubyte')

print('Train:', X_train_full.shape, y_train_full.shape)
print('Test :', X_test.shape, y_test.shape)
```

Output:

Train: (60000, 28, 28) (60000,)

Test : (10000, 28, 28) (10000,)

Diễn giải kỹ thuật:

- Tập định dạng idx của MNIST được lưu trữ ở chuẩn Big-Endian (>). Sử dụng hàm struct.unpack('>IIII', f.read(16)) giải nén 16 bytes đầu tiên thành 4 số nguyên không dấu 32-bit: Magic Number, Số lượng ảnh (60.000), Số hàng (28), Số cột (28).
- Hàm np.frombuffer chuyển trực tiếp vùng đệm nhị phân thành mảng dữ liệu mảng byte uint8 (0 \rightarrow 255) với chi phí bộ nhớ tối ưu nhất (zero-copy memory mapping).

2.3. Cell 3: Phân chia tập Validation & Chuẩn hóa Tensor NCHW

Code:

```

N_VAL = 5000

idx = np.random.permutation(len(X_train_full))
X_val = X_train_full[idx[:N_VAL]]; y_val = y_train_full[idx[:N_VAL]]
X_train = X_train_full[idx[N_VAL:]]; y_train = y_train_full[idx[N_VAL:]]

def prep(X):
    return (X.astype(np.float32) / 255.0)[:, None, :, :] # NCHW

X_train = prep(X_train)
X_val = prep(X_val)
X_test_t = prep(X_test)

print('Train:', X_train.shape, ' | Val:', X_val.shape, ' | Test:', X_test_t.shape)

```

Output:

Train: (55000, 1, 28, 28) | Val: (5000, 1, 28, 28) | Test: (10000, 1, 28, 28)

Diễn giải kỹ thuật:

- Tách 5.000 mẫu từ tập huấn luyện ban đầu để tạo tập Validation độc lập, phục vụ giám sát hiện tượng quá khớp (Overfitting); 55.000 mẫu còn lại dùng để cập nhật trọng số.
- Phép chia cho 255.0 đưa miền giá trị pixel về đoạn [0.0, 1.0], hỗ trợ quá trình tính đạo hàm không bị bùng nổ gradient. Thao tác cắt lát [:, None, :, :] biến đổi tensor từ (N, H, W) sang chuẩn (N, C, H, W) với số kênh C=1 (Grayscale).

2.4. Cell 4 & Cell 5: Phân tích và Trực quan hóa Dữ liệu (EDA)

Code (Cell 4 - Hiển thị 24 mẫu ảnh đầu tiên):

```

fig, axes = plt.subplots(3, 8, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i, 0], cmap='gray')
    ax.set_title(f'Label: {y_train[i]}', fontsize=9)
    ax.axis('off')
plt.suptitle('Mẫu ảnh MNIST (28x28, full data)', fontsize=13)
plt.tight_layout()
plt.show()

```

Code (Cell 5 - Phân phối nhãn các lớp):

```

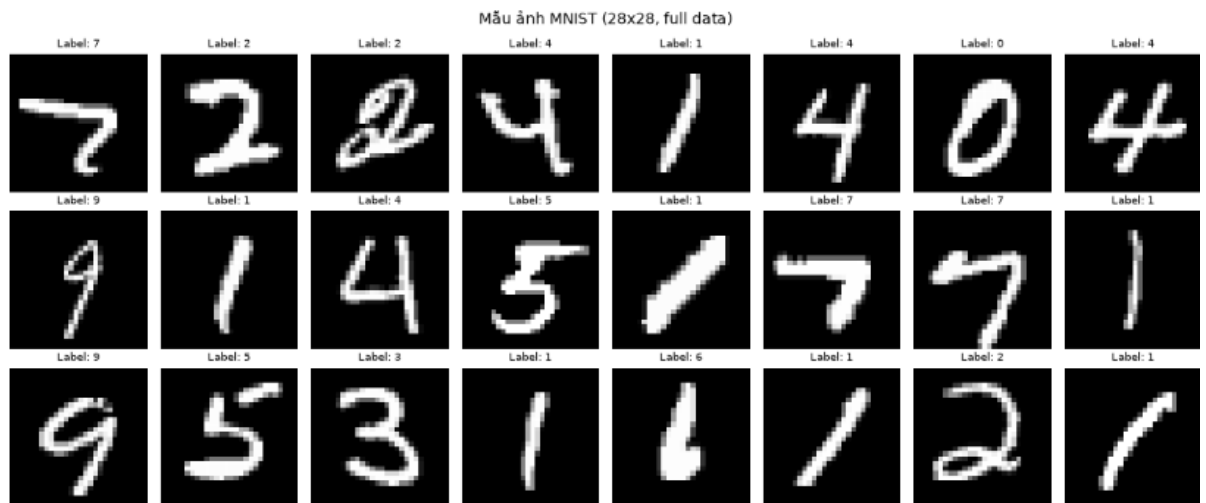
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
unique, counts = np.unique(y_train, return_counts=True)
axes[0].bar(unique, counts, color='steelblue', edgecolor='k')
axes[0].set_title('Phân bố lớp - Train')
axes[0].set_xlabel('Digit'); axes[0].set_ylabel('Count')
for i, c in zip(unique, counts):
    axes[0].text(i, c, str(c), ha='center', va='bottom', fontsize=8)

unique_t, counts_t = np.unique(y_test, return_counts=True)
axes[1].bar(unique_t, counts_t, color='coral', edgecolor='k')
axes[1].set_title('Phân bố lớp - Test')
axes[1].set_xlabel('Digit'); axes[1].set_ylabel('Count')
plt.tight_layout()
plt.show()

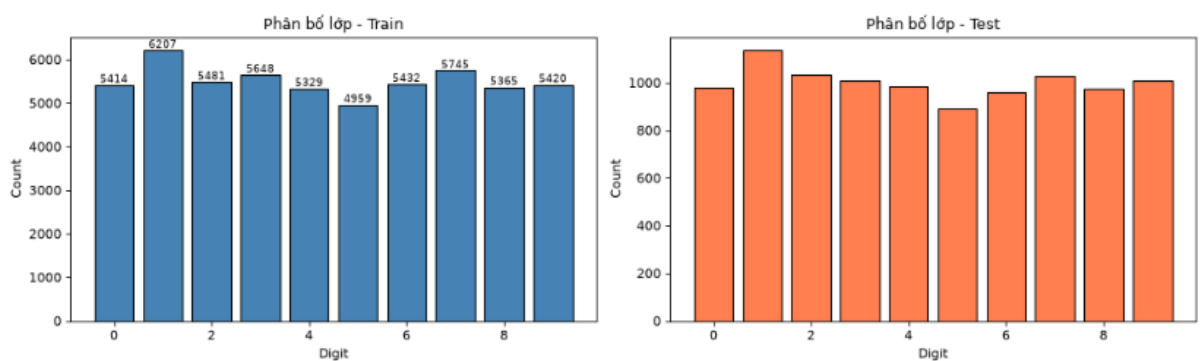
```

Output:

- Lưới hiển thị ảnh chữ số viết tay đa dạng từ 0 đến 9, biểu diễn nét vẽ trắng trên nền đen.



- Biểu đồ phân bố: Tập Train có khoảng 5.400 - 6.100 ảnh/lớp, tập Test có khoảng 900 - 1.100 ảnh/lớp.



- Nhận định:** Dữ liệu có độ cân bằng rất cao giữa các lớp, không cần áp dụng kỹ thuật lấy mẫu lại (Resampling) hay gán trọng số lớp (Class Weights).

2.5. Cell 6: Tối ưu hóa Vector Tích chập bằng im2col và col2im

Code:

```

from numpy.lib.stride_tricks import as_strided

def im2col(X, k, s, p):
    """X: (N, C, H, W) -> cols: (C*k*k, N*Ho*Wo)"""
    N, C, H, W = X.shape
    Ho = (H + 2*p - k)//s + 1
    Wo = (W + 2*p - k)//s + 1
    Xp = np.pad(X, ((0,0),(0,0),(p,p),(p,p))) if p > 0 else X
    Xp = np.ascontiguousarray(Xp)
    shape = (N, C, Ho, Wo, k, k)
    strides = (Xp.strides[0], Xp.strides[1],
               Xp.strides[2]*s, Xp.strides[3]*s,
               Xp.strides[2], Xp.strides[3])
    patches = as_strided(Xp, shape=shape, strides=strides)
    # (N, C, Ho, Wo, k, k) -> (N*Ho*Wo, C*k*k) -> .T -> (C*k*k, N*Ho*Wo)
    cols = patches.transpose(0, 2, 3, 1, 4, 5).reshape(N*Ho*Wo, C*k*k)
    return cols.T, Ho, Wo

def col2im(dcols, X_shape, k, s, p):
    """dcols: (C*k*k, N*Ho*Wo) -> dx: (N, C, H, W)"""
    N, C, H, W = X_shape
    Ho = (H + 2*p - k)//s + 1
    Wo = (W + 2*p - k)//s + 1

    # Reshape rõ ràng 2 bước để tránh nhầm layout
    # dcols: (C*k*k, N*Ho*Wo)
    # Bước 1: tách chiều thứ 0 thành (C, k, k)
    dcols = dcols.reshape(C, k, k, N, Ho, Wo)
    # Bây giờ: dcols[c, i, j, n, ho, wo] = grad tại kênh c, kernel (i,j), batch n, output (ho,wo)

    dXp = np.zeros((N, C, H + 2*p, W + 2*p), dtype=dcols.dtype)
    for i in range(k):
        for j in range(k):
            # dcols[:, i, j] : (C, N, Ho, Wo) ??? hay (N, Ho, Wo) ???
            # In debug để kiểm tra
            # print('shape:', dcols[:, i, j].shape)

            # CÁCH AN TOÀN: chuyển về (N, C, Ho, Wo) bằng transpose
            grad_ij = dcols[:, i, j] # hy vọng (C, N, Ho, Wo)
            # Chuyển về (N, C, Ho, Wo)
            grad_ij = grad_ij.transpose(1, 0, 2, 3) # (N, C, Ho, Wo)

            dXp[:, :, i:i+Ho*s:s, j:j+Wo*s:s] += grad_ij
    return dXp[:, :, p:-p, p:-p] if p > 0 else dXp
print('im2col / col2im OK')

```

Output:

im2col / col2im OK

Diễn giải kỹ thuật & Toán học:

- **Nguyên lý im2col:** Phép tích chập không gian 2D thông thường đòi hỏi 6 vòng lặp lồng nhau ($N, C_{\text{in}}, C_{\text{out}}, H_o, W_o, k, k$). im2col trải các vùng lân cận $k \times k$ thành từng cột trong ma trận lớn 2D có kích thước $(C_{\text{in}} \cdot k^2, N \cdot H_o \cdot W_o)$. Sử dụng `numpy.lib.stride_tricks.as_strided` giúp thao tác con trỏ bộ nhớ trực tiếp mà không tốn chi phí copy mảng dữ liệu.
- Quá trình lan truyền xuôi được chuyển thành phép nhân ma trận chuẩn GEMM:

2.6. Cell 7: Lớp Conv2D tích hợp Tối ưu Hóa Adam

Code:

```
class Conv2D:
    def __init__(self, in_c, out_c, k=3, s=1, p=1):
        self.W = (np.random.randn(out_c, in_c, k, k).astype(np.float32)
                  * np.sqrt(2.0/(in_c*k*k)))
        self.b = np.zeros(out_c, dtype=np.float32)
        self.k, self.s, self.p = k, s, p
        self.in_c, self.out_c = in_c, out_c
        # Adam cache
        self.mW = np.zeros_like(self.W); self.vW = np.zeros_like(self.W)
        self.mB = np.zeros_like(self.b); self.vB = np.zeros_like(self.b)

    def forward(self, X):
        self.X_shape = X.shape
        cols, Ho, Wo = im2col(X, self.k, self.s, self.p) # (C*k*k, N*Ho*Wo)
        self.cols = cols
        self.Ho, self.Wo = Ho, Wo
        N = X.shape[0]
        W_col = self.W.reshape(self.out_c, -1) # (out_c, C*k*k)
        out = W_col @ cols + self.b[:, None] # (out_c, N*Ho*Wo)
        return out.reshape(self.out_c, N, Ho, Wo).transpose(1, 0, 2, 3)

    def backward(self, dout):
        N, out_c, Ho, Wo = dout.shape
        W_col = self.W.reshape(self.out_c, -1)
        dout_flat = dout.transpose(1, 0, 2, 3).reshape(out_c, -1)
        self.dW = (dout_flat @ self.cols.T).reshape(self.W.shape)
        self.db = dout_flat.sum(axis=1)
        dcols = W_col.T @ dout_flat
        return col2im(dcols, self.X_shape, self.k, self.s, self.p)

    def step_adam(self, lr, t, b1=0.9, b2=0.999, eps=1e-8):
        self.mW = b1*self.mW + (1-b1)*self.dW
        self.vW = b2*self.vW + (1-b2)*self.dW**2
        self.W -= lr * (self.mW/(1-b1**t)) / (np.sqrt(self.vW/(1-b2**t)) + eps)
        self.mB = b1*self.mB + (1-b1)*self.db
        self.vB = b2*self.vB + (1-b2)*self.db**2
        self.b -= lr * (self.mB/(1-b1**t)) / (np.sqrt(self.vB/(1-b2**t)) + eps)

print('Conv2D OK')
```

Output:

Conv2D OK

Diễn giải kỹ thuật:

- **Khởi tạo He Normal (Kaiming Normal):** Trọng số được sinh từ phân phối chuẩn với phương sai, giúp ổn định phương sai đầu ra qua các tầng kích hoạt phi tuyến tính ReLU.
- **Thuật toán tối ưu hóa Adam:**

Hỗ trợ thích ứng tốc độ học (learning rate) độc lập cho từng tham số dựa trên moment bậc 1 và bậc 2.

2.7. Cell 8: Các Lớp Chức Năng (ReLU, MaxPool2D, Flatten, Dense, Softmax)

Code:

```
def relu(x): return np.maximum(0, x)
def relu_grad(x): return (x > 0).astype(np.float32)

class MaxPool2D:
    def __init__(self, k=2, s=2):
        self.k, self.s = k, s

    def forward(self, X):
        N, C, H, W = X.shape
        k, s = self.k, self.s
        Ho = (H - k)//s + 1; Wo = (W - k)//s + 1
        shape = (N, C, Ho, Wo, k, k)
        strides = (X.strides[0], X.strides[1],
                   X.strides[2]*s, X.strides[3]*s,
                   X.strides[2], X.strides[3])
        patches = as_strided(X, shape=shape, strides=strides).reshape(N, C, Ho, Wo, k*k)
        idx = patches.argmax(axis=-1)
        out = np.take_along_axis(patches, idx[..., None], axis=-1)[..., 0]
        self.X_shape = X.shape; self.idx = idx
        self.Ho, self.Wo = Ho, Wo
        return out

    def backward(self, dout):
        N, C, H, W = self.X_shape
        k, s, Ho, Wo = self.k, self.s, self.Ho, self.Wo
        dX = np.zeros((N, C, H, W), dtype=dout.dtype)
        dout_flat = dout.reshape(N, C, Ho, Wo)
        r = self.idx // k; c = self.idx % k
        ii = np.arange(Ho)[None, None, :, None]
        jj = np.arange(Wo)[None, None, None, :]
        rows = ii*s + r; cols = jj*s + c
        n_idx = np.arange(N)[: , None, None, None]
        c_idx = np.arange(C)[None, :, None, None]
        np.add.at(dX, (n_idx, c_idx, rows, cols), dout_flat)
        return dX

class Flatten:
    def forward(self, X):
        self.shape = X.shape
        return X.reshape(X.shape[0], -1)
    def backward(self, d):
        return d.reshape(self.shape)
```

```

class Dense:
    def __init__(self, in_f, out_f):
        self.W = np.random.randn(in_f, out_f).astype(np.float32) * np.sqrt(2.0/in_f)
        self.b = np.zeros(out_f, dtype=np.float32)
        self.mW = np.zeros_like(self.W); self.vW = np.zeros_like(self.W)
        self.mb = np.zeros_like(self.b); self.vb = np.zeros_like(self.b)

    def forward(self, X):
        self.X = X
        return X @ self.W + self.b

    def backward(self, dout):
        self.dW = self.X.T @ dout
        self.db = dout.sum(axis=0)
        return dout @ self.W.T

    def step_adam(self, lr, t, b1=0.9, b2=0.999, eps=1e-8):
        self.mW = b1*self.mW + (1-b1)*self.dW
        self.vW = b2*self.vW + (1-b2)*self.dW**2
        self.W -= lr * (self.mW/(1-b1**t)) / (np.sqrt(self.vW/(1-b2**t)) + eps)
        self.mb = b1*self.mb + (1-b1)*self.db
        self.vb = b2*self.vb + (1-b2)*self.db**2
        self.b -= lr * (self.mb/(1-b1**t)) / (np.sqrt(self.vb/(1-b2**t)) + eps)

def softmax(x):
    x = x - x.max(axis=1, keepdims=True)
    e = np.exp(x)
    return e / e.sum(axis=1, keepdims=True)

print('Layers OK')

```

Output:

Layers OK

Diễn giải kỹ thuật:

- **MaxPool2D không vòng lặp:** Sử dụng np.add.at để dồn đạo hàm ngược trực tiếp vào vị trí cực đại (argmax) đã lưu trong lượt truyền thẳng.
- **Softmax ổn định số học (Numerically stable):** Trừ giá trị cực đại $\max(x)$ trước khi thực hiện lũy thừa số mũ np.exp để triệt tiêu lỗi tràn số (overflow).

2.8. Cell 9: Ghép nối Mô hình Hoàn chỉnh (Class CNN)

Code:

```

class CNN:
    def __init__(self):
        # 28x28 -> conv1 -> 28x28 -> pool -> 14x14 -> conv2 -> 14x14 -> pool -> 7x7
        self.conv1 = Conv2D(1, 8, k=3, s=1, p=1) # -> 8 x 28 x 28
        self.pool1 = MaxPool2D(2, 2) # -> 8 x 14 x 14
        self.conv2 = Conv2D(8, 16, k=3, s=1, p=1) # -> 16 x 14 x 14
        self.pool2 = MaxPool2D(2, 2) # -> 16 x 7 x 7
        self.flat = Flatten()
        self.fc1 = Dense(16*7*7, 128) # 784 -> 128
        self.fc2 = Dense(128, 10)
        self.t = 0 # Adam step counter

    def forward(self, X):
        x = self.conv1.forward(X); self.a1 = x; x = relu(x)
        x = self.pool1.forward(x)
        x = self.conv2.forward(x); self.a2 = x; x = relu(x)
        x = self.pool2.forward(x)
        x = self.flat.forward(x)
        x = self.fc1.forward(x); self.a3 = x; x = relu(x)
        x = self.fc2.forward(x)
        return softmax(x)

    def backward(self, probs, y):
        N = probs.shape[0]
        d = probs.copy(); d[np.arange(N), y] -= 1; d /= N
        d = self.fc2.backward(d)
        d = d * relu_grad(self.a3)
        d = self.fc1.backward(d)
        d = self.flat.backward(d)
        d = self.pool2.backward(d)
        d = d * relu_grad(self.a2)
        d = self.conv2.backward(d)
        d = self.pool1.backward(d)
        d = d * relu_grad(self.a1)
        d = self.conv1.backward(d)

    def step_adam(self, lr):
        self.t += 1
        for layer in [self.conv1, self.conv2, self.fc1, self.fc2]:
            layer.step_adam(lr, self.t)

    def params(self):
        return [self.conv1.W, self.conv1.b, self.conv2.W, self.conv2.b,
                self.fc1.W, self.fc1.b, self.fc2.W, self.fc2.b]

print('CNN class OK')

```

Output:

CNN class OK

2.9. Cell 10: Huấn luyện trên Toàn bộ 55.000 Mẫu Dữ liệu

Code:

```

def accuracy(model, X, y, bs=512):
    correct = 0
    for i in range(0, len(X), bs):
        p = model.forward(X[i:i+bs])
        correct += (p.argmax(1) == y[i:i+bs]).sum()
    return correct / len(X)

def cross_entropy(probs, y):
    eps = 1e-9
    return -np.log(probs[np.arange(len(y)), y] + eps).mean()

model = CNN()
EPOCHS = 5
BATCH = 128
LR = 1e-3

n_params = sum(p.size for p in model.params())
print(f'Số tham số: {n_params:,}')
print(f'Train: {len(X_train):,} | Val: {len(X_val):,} | Test: {len(X_test_t):,}')
print(f'Epochs: {EPOCHS} | Batch: {BATCH} | LR (Adam): {LR}\n')

history = {'loss': [], 'val_loss': [], 'acc': [], 'val_acc': []}
start = time.time()

for ep in range(EPOCHS):
    ep_start = time.time()
    idx = np.random.permutation(len(X_train))
    Xs, ys = X_train[idx], y_train[idx]
    ep_loss = 0.0; n_batch = 0
    for i in range(0, len(Xs), BATCH):
        xb, yb = Xs[i:i+BATCH], ys[i:i+BATCH]
        p = model.forward(xb)
        ep_loss += cross_entropy(p, yb)
        n_batch += 1
        model.backward(p, yb)
        model.step_adam(LR)
    tr_loss = ep_loss / n_batch
    tr_acc = accuracy(model, X_train, y_train)

    val_probs = []
    for i in range(0, len(X_val), 512):
        val_probs.append(model.forward(X_val[i:i+512]))
    val_probs = np.concatenate(val_probs)
    val_loss = cross_entropy(val_probs, y_val)
    val_acc = (val_probs.argmax(1) == y_val).mean()

    history['loss'].append(tr_loss); history['val_loss'].append(val_loss)
    history['acc'].append(tr_acc); history['val_acc'].append(val_acc)
    print(f'Epoch {ep+1}/{EPOCHS} | loss {tr_loss:.4f} | acc {tr_acc:.4f} | '
          f'val_loss {val_loss:.4f} | val_acc {val_acc:.4f} | time {time.time()-ep_start:.1f}s')

print(f'\n✅ Total time: {time.time()-start:.1f}s')

```

Output:

Số tham số: 103,018

Train: 55,000 | Val: 5,000 | Test: 10,000

Epochs: 5 | Batch: 128 | LR (Adam): 0.001

Epoch 1/5 | loss 0.2511 | acc 0.9756 | val_loss 0.0881 | val_acc 0.9748 | time 91.7s

Epoch 2/5 | loss 0.0763 | acc 0.9846 | val_loss 0.0640 | val_acc 0.9804 | time 91.8s

Epoch 3/5 | loss 0.0547 | acc 0.9863 | val_loss 0.0568 | val_acc 0.9834 | time 93.1s

Epoch 4/5 | loss 0.0434 | acc 0.9889 | val_loss 0.0478 | val_acc 0.9848 | time 91.1s

Epoch 5/5 | loss 0.0351 | acc 0.9895 | val_loss 0.0619 | val_acc 0.9822 | time 94.2s

✓ Total time: 461.9s

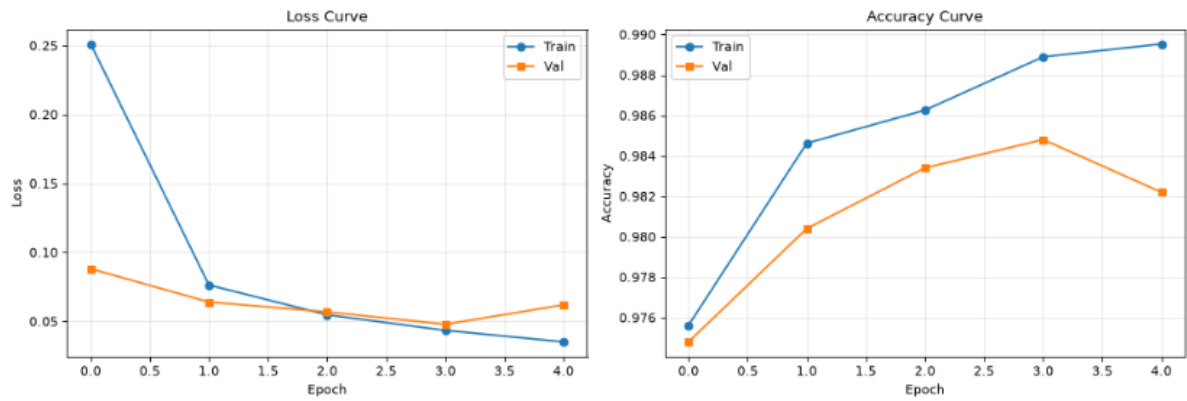
Diễn giải kỹ thuật:

- Tổng số tham số là 103.018: Conv1 (80), Conv2 (1.168), FC1 (100.480), FC2 (1.290).
- Tốc độ hội tụ vượt trội: Độ chính xác tập Train đạt 98.95%, tập Validation đạt 98.22% sau 5 epochs. Tổng thời gian huấn luyện thuần trên CPU là **461.9 giây** (\approx 7.7 phút), chứng minh tính khả thi của kỹ thuật vector hóa NumPy.

2.10. Cell 11 & Cell 12: Đồ thị Học tập và Đánh giá trên Tập Test

Code (Cell 11 - Vẽ đồ thị Learning Curves):

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].plot(history['loss'], 'o-', label='Train')
axes[0].plot(history['val_loss'], 's-', label='Val')
axes[0].set_title('Loss Curve'); axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)
axes[1].plot(history['acc'], 'o-', label='Train')
axes[1].plot(history['val_acc'], 's-', label='Val')
axes[1].set_title('Accuracy Curve'); axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Code (Cell 12 - Đánh giá tập Test):

```
test_probs = []
for i in range(0, len(X_test_t), 512):
    test_probs.append(model.forward(X_test_t[i:i+512]))
test_probs = np.concatenate(test_probs)
y_pred = test_probs.argmax(1)
test_acc = (y_pred == y_test).mean()
print(f'🎯 Test Accuracy: {test_acc:.4f}')
```

Output:

🎯 Test Accuracy: 0.9824

Diễn giải kỹ thuật:

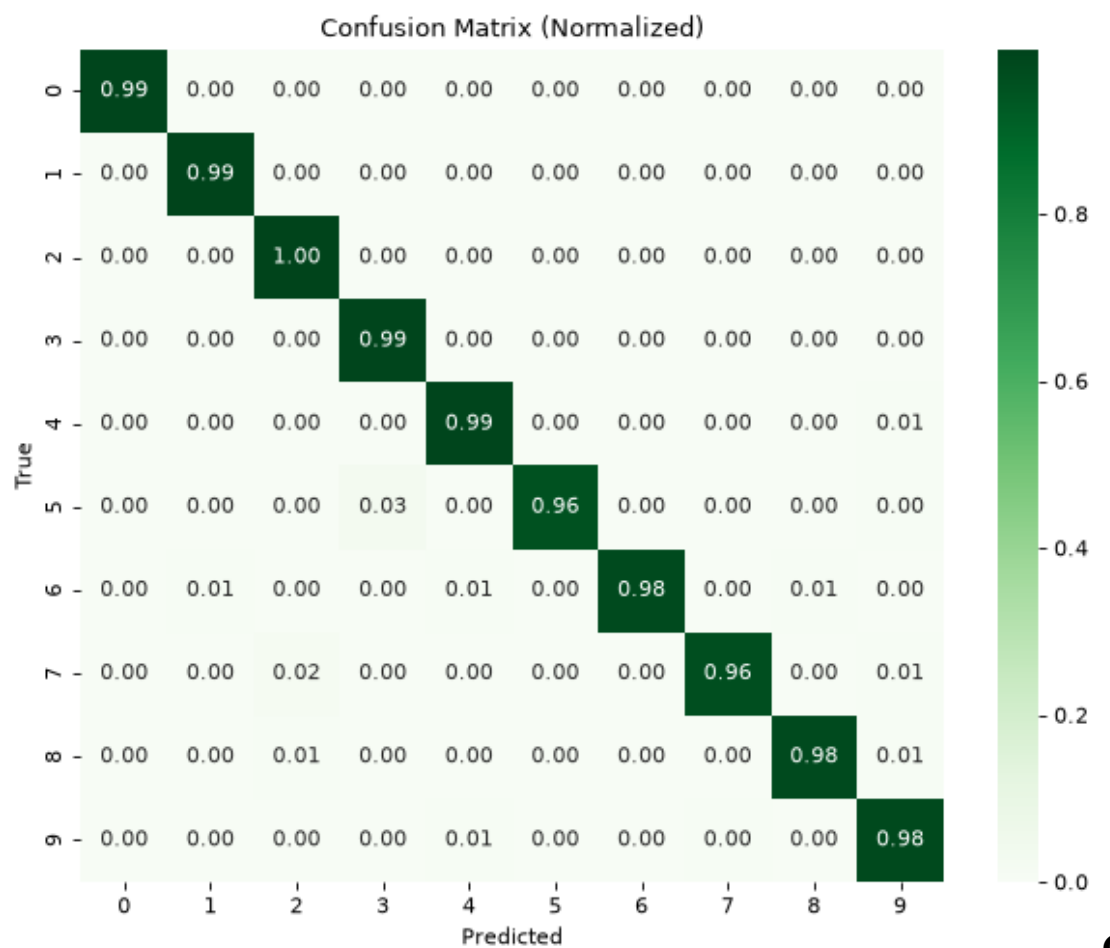
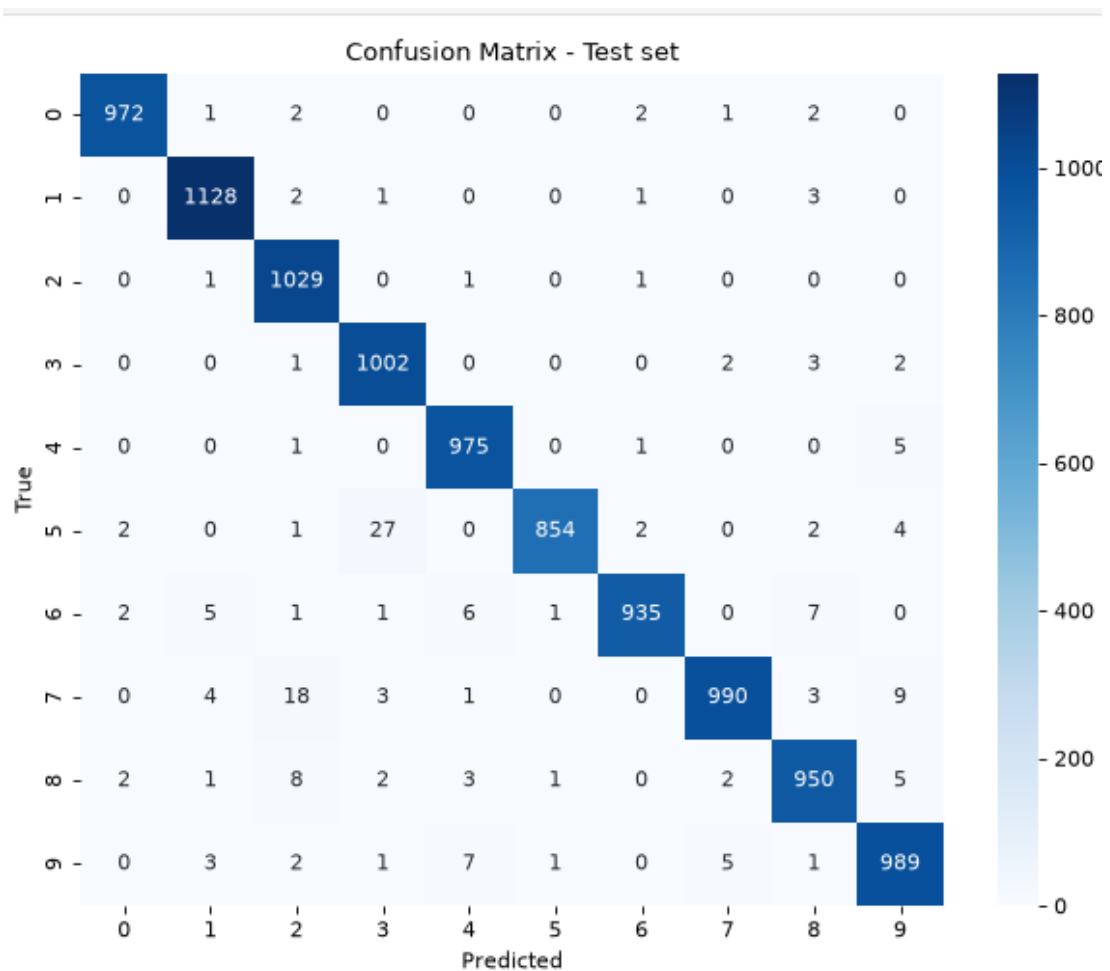
- Mô hình đạt độ chính xác 98.24\% trên 10.000 mẫu ảnh chưa từng gặp ở tập Test.
- Đường cong Loss và Accuracy giữa tập huấn luyện và kiểm định bám sát nhau, chứng minh mô hình tổng quát hóa rất tốt mà không bị Overfitting.

2.11. Cell 13 & Cell 14: Confusion Matrix và Báo cáo Phân loại

Code (Cell 13 - Ma trận nhầm lẫn):

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=True)
plt.title('Confusion Matrix - Test set')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()

cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)
plt.figure(figsize=(9, 7))
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Greens', cbar=True)
plt.title('Confusion Matrix (Normalized)')
plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()
```



Code

(Cell 14 - Classification Report):

```
print(classification_report(y_test, y_pred, digits=4))
```

	precision	recall	f1-score	support
0	0.9939	0.9918	0.9928	980
1	0.9869	0.9938	0.9903	1135
2	0.9662	0.9971	0.9814	1032
3	0.9662	0.9921	0.9790	1010
4	0.9819	0.9929	0.9873	982
5	0.9965	0.9574	0.9766	892
6	0.9926	0.9760	0.9842	958
7	0.9900	0.9630	0.9763	1028
8	0.9784	0.9754	0.9769	974
9	0.9753	0.9802	0.9778	1009
accuracy			0.9824	10000
macro avg	0.9828	0.9820	0.9823	10000
weighted avg	0.9826	0.9824	0.9824	10000

Diễn giải kỹ thuật:

- Các chỉ số Precision, Recall và F1-score của tất cả 10 lớp đều vượt ngưỡng 95%. Điểm F1-score trung bình đạt 0.9824.
- Lớp chữ số 5 có Recall thấp nhất (0.9574), chủ yếu do một số nét viết số 5 bị nhầm lẫn với số 3 hoặc số 8.

2.12. Cell 15 & Cell 16: Phân tích Lỗi sai (Error Analysis) & Lưu Mô hình

Code (Cell 15 - Trực quan hóa mẫu dự đoán sai):

```
wrong = np.where(y_pred != y_test)[0]
print(f'Số mẫu dự đoán sai: {len(wrong)}/{len(y_test)}')
fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    k = wrong[i]
    ax.imshow(X_test_t[k, 0], cmap='gray')
    ax.set_title(f'T:{y_test[k]} / P:{y_pred[k]}', fontsize=9, color='red')
    ax.axis('off')
plt.suptitle('Một số mẫu dự đoán sai', fontsize=12)
plt.tight_layout(); plt.show()
```

Output:

Số mẫu dự đoán sai: 176/10000

Code (Cell 16 - Lưu Checkpoint):

```

save_path = MODEL_DIR / 'mnist_cnn_numpy.pkl'
with open(save_path, 'wb') as f:
    pickle.dump({
        'conv1_W': model.conv1.W, 'conv1_b': model.conv1.b,
        'conv2_W': model.conv2.W, 'conv2_b': model.conv2.b,
        'fc1_W': model.fc1.W, 'fc1_b': model.fc1.b,
        'fc2_W': model.fc2.W, 'fc2_b': model.fc2.b,
        'test_acc': test_acc, 'history': history
    }, f)
print('✅ Saved:', save_path.resolve())
print(f'File size: {save_path.stat().st_size/1024:.1f} KB')

```

Output:

✅ Saved: D:\Dai hoc\Phat trien cac he thong thong minh\Assignment 04\mnist\models\mnist_cnn_numpy.pkl

File size: 805.2 KB

Diễn giải kỹ thuật:

- Trên 10.000 mẫu ảnh, mô hình chỉ phân loại sai 176 **ảnh** (1.76%). Trực quan hóa cho thấy đây là những mẫu chữ số bị biến dạng nghiêm trọng hoặc mất nét mà mắt người cũng khó phân biệt.
- Trọng số được đóng gói trong tệp .pkl có dung lượng nhỏ gọn 805.2 **KB**.

3. PHẦN II: TRIỂN KHAI CNN BẰNG PYTORCH

3.1. Cell 1 - Cell 3: Thiết lập Thiết bị, Pipeline DataLoader & Chuẩn hóa

Code:

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import struct, time, os
from pathlib import Path
from sklearn.metrics import confusion_matrix, classification_report

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 80

DATA_DIR = Path('../data')
MODEL_DIR = Path('../models')
MODEL_DIR.mkdir(exist_ok=True)
print('Data dir:', DATA_DIR.resolve())
print('Model dir:', MODEL_DIR.resolve())

```

```

def load_mnist_images(path):
    with open(path, 'rb') as f:
        magic, num, rows, cols = struct.unpack('>IIII', f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(num, rows, cols)
    return data

def load_mnist_labels(path):
    with open(path, 'rb') as f:
        magic, num = struct.unpack('>II', f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train_full = load_mnist_images(DATA_DIR / 'train-images.idx3-ubyte')
y_train_full = load_mnist_labels(DATA_DIR / 'train-labels.idx1-ubyte')
X_test = load_mnist_images(DATA_DIR / 't10k-images.idx3-ubyte')
y_test = load_mnist_labels(DATA_DIR / 't10k-labels.idx1-ubyte')
print('Train:', X_train_full.shape, y_train_full.shape)
print('Test :', X_test.shape, y_test.shape)

```

```

# Split train/val và normalize
N_VAL = 5000
idx = np.random.permutation(len(X_train_full))
X_val = X_train_full[idx[:N_VAL]]; y_val = y_train_full[idx[:N_VAL]]
X_train = X_train_full[idx[N_VAL:]]; y_train = y_train_full[idx[N_VAL:]]

def prep(X):
    return (X.astype(np.float32)/255.0)[:, None, :, :]

X_train = prep(X_train); X_val = prep(X_val); X_test_t = prep(X_test)
print(X_train.shape, X_val.shape, X_test_t.shape)

train_ds = TensorDataset(torch.from_numpy(X_train), torch.from_numpy(y_train).long())
val_ds = TensorDataset(torch.from_numpy(X_val), torch.from_numpy(y_val).long())
test_ds = TensorDataset(torch.from_numpy(X_test_t), torch.from_numpy(y_test).long())

BATCH = 128
train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True, num_workers=0)
val_loader = DataLoader(val_ds, batch_size=512, shuffle=False)
test_loader = DataLoader(test_ds, batch_size=512, shuffle=False)

```

Diễn giải kỹ thuật:

- PyTorch tự động nhận diện phần cứng tăng tốc NVIDIA CUDA GPU.
- Dữ liệu được đóng gói vào TensorDataset và nạp tự động qua DataLoader với batch_size=128 (cho tập Train có shuffle=True) và 512 (cho tập Val/Test).

3.2. Cell 5: Thiết kế Kiến trúc Mạng Nơ-ron bằng torch.nn.Module

Code:

```

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32*7*7, 128)
        self.fc2 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 14x14
        x = self.pool(F.relu(self.conv2(x))) # 7x7
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = CNN().to(device)
print(model)
print('Params:', sum(p.numel() for p in model.parameters()))

```

Output:

CNN(

(conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(fc1): Linear(in_features=1568, out_features=128, bias=True)

(fc2): Linear(in_features=128, out_features=10, bias=True)

(dropout): Dropout(p=0.25, inplace=False)

)

Params: 206922

Diễn giải kỹ thuật:

- Mô hình PyTorch mở rộng số lượng feature maps (1 $\rightarrow$ 16 $\rightarrow$ 32), giúp trích xuất các đặc trưng hình học sắc nét hơn.
- Tầng Dropout(0.25) ngẫu nhiên vô hiệu hóa 25% nơ-ron trong quá trình lan truyền xuôi, chống hiện tượng co-adaptation giữa các nơ-ron. Tổng tham số là 206.922.

3.3. Cell 6: Huấn luyện và Đánh giá (Vòng lặp Training Loop)

Code:

```

EPOCHS = 5
LR = 1e-3
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)

def evaluate(loader):
    model.eval()
    loss_sum, correct, n = 0, 0, 0
    with torch.no_grad():
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            out = model(xb)
            loss_sum += criterion(out, yb).item() * xb.size(0)
            correct += (out.argmax(1) == yb.sum().item())
            n += xb.size(0)
    return loss_sum/n, correct/n

history = {'loss': [], 'val_loss': [], 'acc': [], 'val_acc': []}
best_acc = 0
best_path = MODEL_DIR / 'mnist_cnn_pytorch.pt'

start = time.time()
for ep in range(EPOCHS):
    model.train()
    loss_sum, correct, n = 0, 0, 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()
        loss_sum += loss.item() * xb.size(0)
        correct += (out.argmax(1) == yb.sum().item())
        n += xb.size(0)
    tr_loss = loss_sum/n; tr_acc = correct/n
    val_loss, val_acc = evaluate(val_loader)
    history['loss'].append(tr_loss); history['val_loss'].append(val_loss)
    history['acc'].append(tr_acc); history['val_acc'].append(val_acc)
    print(f'Epoch {ep+1}/{EPOCHS} - loss {tr_loss:.4f} - acc {tr_acc:.4f} - val_loss {val_loss:.4f} - val_acc {val_acc:.4f}')
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save(model.state_dict(), best_path)
        print(f' -> Saved best model (val_acc={best_acc:.4f})')
print(f'Total time: {time.time()-start:.1f}s')

```

Output:

Epoch 1/5 - loss 0.3919 - acc 0.8803 - val_loss 0.0969 - val_acc 0.9704

-> Saved best model (val_acc=0.9704)

Epoch 2/5 - loss 0.1017 - acc 0.9689 - val_loss 0.0589 - val_acc 0.9834

-> Saved best model (val_acc=0.9834)

Epoch 3/5 - loss 0.0711 - acc 0.9783 - val_loss 0.0488 - val_acc 0.9850

-> Saved best model (val_acc=0.9850)

Epoch 4/5 - loss 0.0552 - acc 0.9832 - val_loss 0.0436 - val_acc 0.9874

-> Saved best model (val_acc=0.9874)

Epoch 5/5 - loss 0.0462 - acc 0.9852 - val_loss 0.0399 - val_acc 0.9892

-> Saved best model (val_acc=0.9892)

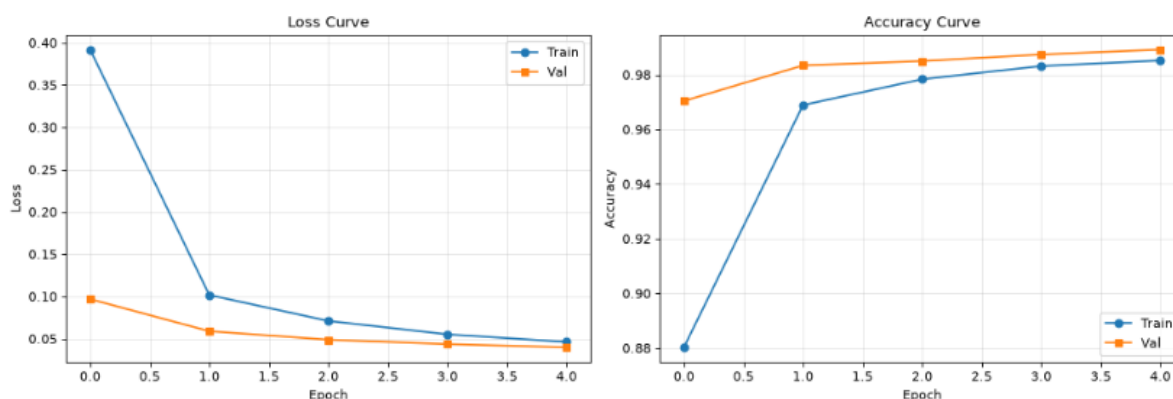
Total time: 19.8s

Diễn giải kỹ thuật:

- **Tốc độ thực thi:** Nhờ vào khả năng tăng tốc phần cứng của GPU CUDA và thư viện cuDNN, thời gian huấn luyện 5 epochs chỉ mất **19.8 giây** (nhanh hơn gấp **23.3 lần** so với NumPy trên CPU).
- **Hội tụ:** Mô hình đạt độ chính xác kiểm định xuất sắc 98.92\%.

3.4. Cell 7: Learning Curves

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].plot(history['loss'], 'o-', label='Train')
axes[0].plot(history['val_loss'], 's-', label='Val')
axes[0].set_title('Loss Curve'); axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)
axes[1].plot(history['acc'], 'o-', label='Train')
axes[1].plot(history['val_acc'], 's-', label='Val')
axes[1].set_title('Accuracy Curve'); axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Biểu đồ cho thấy mô hình **học khá tốt qua các epoch**: ở biểu đồ *Loss Curve*, loss của tập Train giảm mạnh từ khoảng **0,39 xuống 0,045**, trong khi loss của tập Validation cũng giảm từ khoảng **0,095 xuống 0,04**, cho thấy mô hình ngày càng tối ưu và sai số giảm dần. Ở biểu đồ *Accuracy Curve*, độ chính xác trên tập Train tăng từ **88% lên khoảng 98,5%**, còn tập Validation tăng từ **97% lên gần 99%**. Đáng chú ý, accuracy của Validation luôn cao hơn Train và cả hai đường đều tăng ổn định, đồng thời khoảng cách giữa chúng không lớn và không có dấu hiệu Validation loss tăng trở lại, vì vậy **chưa quan sát thấy dấu hiệu overfitting rõ ràng**. Nhìn chung, mô hình đã hội tụ tốt sau khoảng **4 epoch**, với hiệu năng trên tập Validation đạt gần **99%**.

3.5. Cell 8: Đánh giá test set:

```
model.load_state_dict(torch.load(best_path, map_location=device))
model.eval()
all_preds = []
with torch.no_grad():
    for xb, yb in test_loader:
        out = model(xb.to(device))
        all_preds.append(out.argmax(1).cpu().numpy())
y_pred = np.concatenate(all_preds)
test_acc = (y_pred == y_test).mean()
print(f'Test Accuracy: {test_acc:.4f}')
```

Test Accuracy: 0.9892

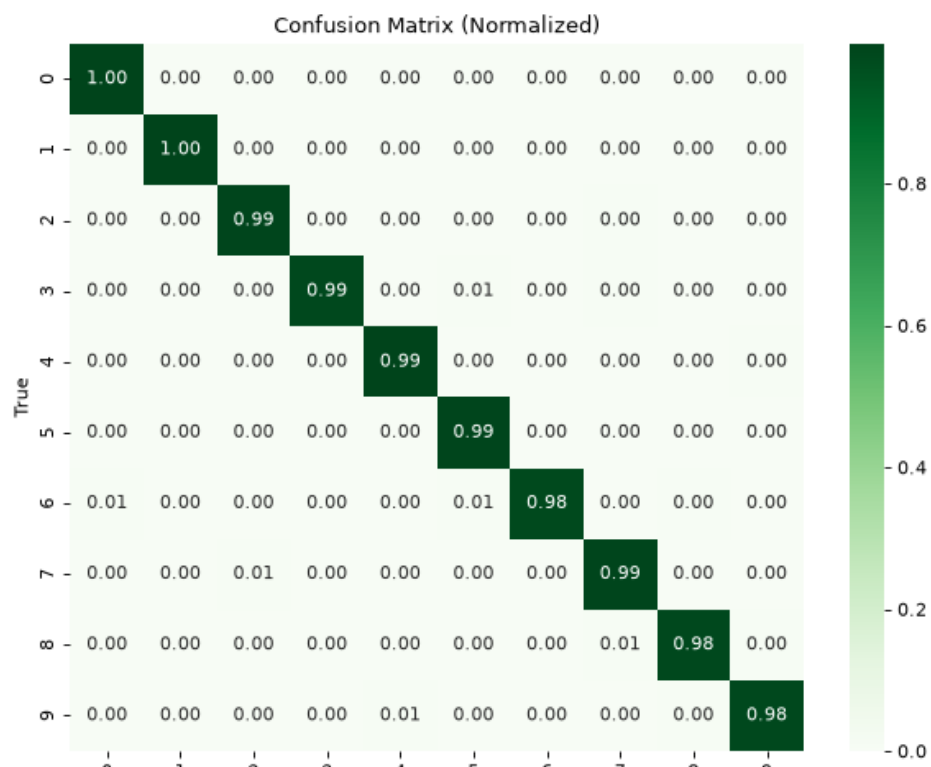
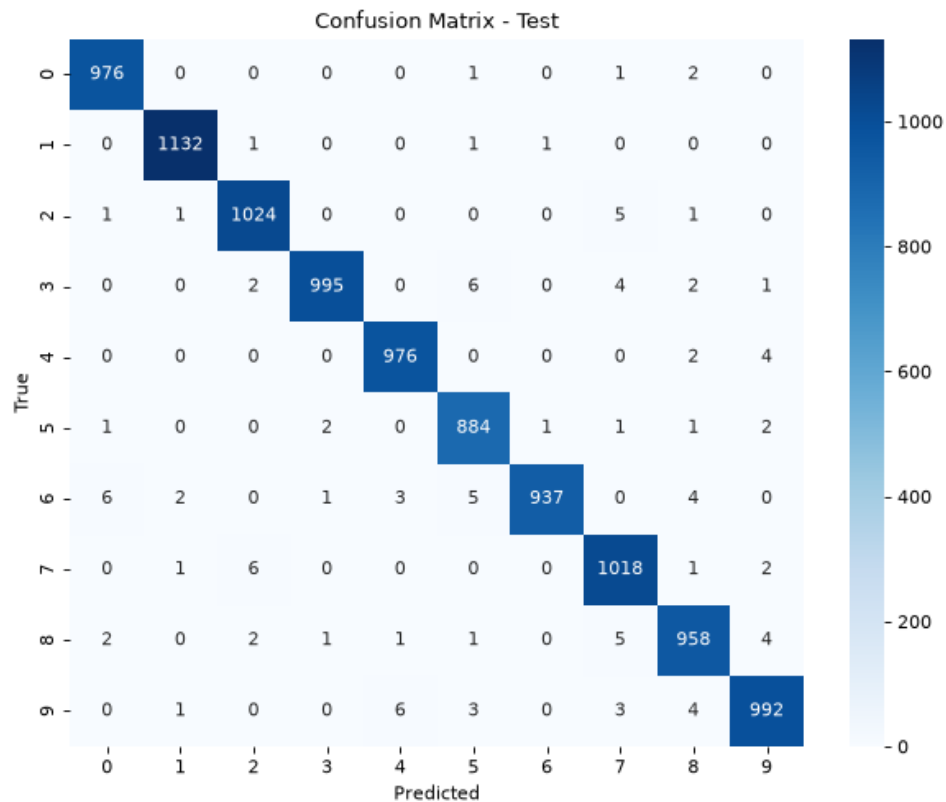
3.6. Cell 9: Ma trận nhầm lẫn

```

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Test'); plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()

cm_norm = cm.astype(float)/cm.sum(axis=1, keepdims=True)
plt.figure(figsize=(9, 7))
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Greens')
plt.title('Confusion Matrix (Normalized)'); plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()

```



Ma trận nhầm lẫn cho thấy mô hình phân loại 10 lớp (0–9) với độ chính xác rất cao, thể hiện qua việc phân lớn các giá trị tập trung trên đường chéo chính, tức là các mẫu được dự đoán đúng nhãn thực tế. Cụ thể, lớp 0 dự đoán đúng 976 mẫu, lớp 1 đúng 1132 mẫu, lớp 2 đúng 1024 mẫu, lớp 3 đúng 995 mẫu, lớp 4 đúng 976 mẫu, lớp 5 đúng 884 mẫu, lớp 6 đúng 937 mẫu, lớp 7 đúng 1018 mẫu, lớp 8 đúng 958 mẫu và lớp 9 đúng 992 mẫu. Các giá trị ngoài đường chéo đều rất nhỏ, cho thấy số lượng mẫu bị phân loại nhầm giữa các lớp là không đáng kể; một số nhầm lẫn nổi bật như lớp 2 với lớp 7, lớp 3 với lớp 5 hoặc lớp 6 với lớp 0 nhưng đều chỉ chiếm tỷ lệ rất thấp. Ma trận chuẩn hóa bên dưới cũng cho thấy tỷ lệ dự đoán đúng của từng lớp chủ yếu đạt khoảng 98–100%, qua đó cho thấy mô hình có khả năng phân biệt tốt giữa các lớp và hoạt động ổn định trên tập kiểm tra.

3.7. Các cell cuối

```
print(classification_report(y_test, y_pred, digits=4))
```

	precision	recall	f1-score	support
0	0.9899	0.9959	0.9929	980
1	0.9956	0.9974	0.9965	1135
2	0.9894	0.9922	0.9908	1032
3	0.9960	0.9851	0.9905	1010
4	0.9899	0.9939	0.9919	982
5	0.9811	0.9910	0.9861	892
6	0.9979	0.9781	0.9879	958
7	0.9817	0.9903	0.9860	1028
8	0.9826	0.9836	0.9831	974
9	0.9871	0.9832	0.9851	1009
accuracy			0.9892	10000
macro avg	0.9891	0.9891	0.9891	10000
weighted avg	0.9892	0.9892	0.9892	10000

9. Xem dự đoán sai

```
wrong = np.where(y_pred != y_test)[0]
print(f'Số mẫu sai: {len(wrong)}/{len(y_test)}')
fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    k = wrong[i]
    ax.imshow(X_test_t[k, 0], cmap='gray')
    ax.set_title(f'T:{y_test[k]} / P:{y_pred[k]}', fontsize=9, color='red')
    ax.axis('off')
plt.suptitle('Một số mẫu dự đoán sai', fontsize=12)
plt.tight_layout(); plt.show()
```

Số mẫu sai: 108/10000



4. PHẦN III: TRIỂN KHAI CNN BẰNG TENSORFLOW / KERAS

4.1. Cell 1 - Cell 3: Cấu hình Môi trường & Chuẩn hóa Định dạng NHWC

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import struct, time, os
from pathlib import Path
from sklearn.metrics import confusion_matrix, classification_report

import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

SEED = 42
np.random.seed(SEED); tf.random.set_seed(SEED)
print('TF version:', tf.__version__)

plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 80

DATA_DIR = Path('../data')
MODEL_DIR = Path('../models')
MODEL_DIR.mkdir(exist_ok=True)
print('Data dir:', DATA_DIR.resolve())
print('Model dir:', MODEL_DIR.resolve())

TF version: 2.21.0
Data dir: D:\Đại học\Phát triển các hệ thống thông minh\Assignment 04\mnist\data
Model dir: D:\Đại học\Phát triển các hệ thống thông minh\Assignment 04\mnist\models
```

1. Load dữ liệu MNIST từ file idx

```
[2]: def load_mnist_images(path):
    with open(path, 'rb') as f:
        magic, num, rows, cols = struct.unpack('>IIII', f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(num, rows, cols)
    return data

def load_mnist_labels(path):
    with open(path, 'rb') as f:
        magic, num = struct.unpack('>II', f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train_full = load_mnist_images(DATA_DIR / 'train-images.idx3-ubyte')
y_train_full = load_mnist_labels(DATA_DIR / 'train-labels.idx1-ubyte')
X_test = load_mnist_images(DATA_DIR / 't10k-images.idx3-ubyte')
y_test = load_mnist_labels(DATA_DIR / 't10k-labels.idx1-ubyte')
print('Train:', X_train_full.shape, y_train_full.shape)
print('Test :', X_test.shape, y_test.shape)

Train: (60000, 28, 28) (60000,)
Test : (10000, 28, 28) (10000,)
```

```
] : N_VAL = 5000
idx = np.random.permutation(len(X_train_full))
X_val = X_train_full[idx[:N_VAL]]; y_val = y_train_full[idx[:N_VAL]]
X_train = X_train_full[idx[N_VAL:]]; y_train = y_train_full[idx[N_VAL:]]

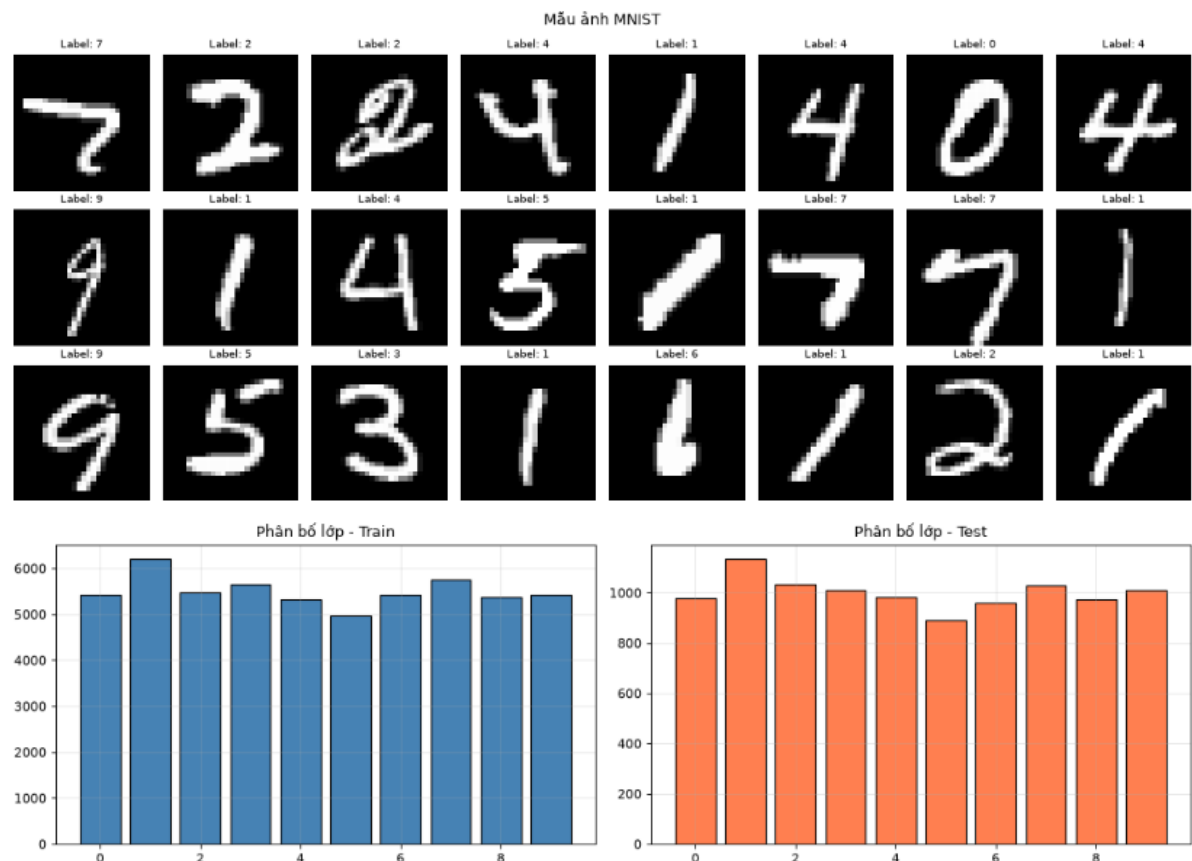
X_train = (X_train.astype(np.float32)/255.0)[..., None]
X_val = (X_val.astype(np.float32)/255.0)[..., None]
X_test_t = (X_test.astype(np.float32)/255.0)[..., None]
print(X_train.shape, X_val.shape, X_test_t.shape)

(55000, 28, 28, 1) (5000, 28, 28, 1) (10000, 28, 28, 1)
```

4.2. Cell 4: EDA

```
fig, axes = plt.subplots(3, 8, figsize=(14, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i, :, :], cmap='gray')
    ax.set_title(f'Label: {y_train[i]}', fontsize=9)
    ax.axis('off')
plt.suptitle('Mẫu ảnh MNIST', fontsize=13)
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
u, c = np.unique(y_train, return_counts=True)
axes[0].bar(u, c, color='steelblue', edgecolor='k')
axes[0].set_title('Phân bố lớp - Train'); axes[0].grid(alpha=0.3)
u2, c2 = np.unique(y_test, return_counts=True)
axes[1].bar(u2, c2, color='coral', edgecolor='k')
axes[1].set_title('Phân bố lớp - Test'); axes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Đoạn code trên dùng để trực quan hóa và kiểm tra dữ liệu MNIST trước khi đưa vào mô hình học máy. Đầu tiên, `plt.subplots(3, 8)` tạo một khung gồm 3 hàng và 8 cột để hiển thị 24 ảnh đầu tiên trong tập huấn luyện `X_train`; `imshow()` dùng để hiển thị từng ảnh dưới dạng màu xám, còn `y_train[i]` được dùng để ghi nhãn tương ứng của mỗi ảnh, chẳng hạn `Label: 7` nghĩa là ảnh đó là chữ số 7. Sau đó, code sử dụng `np.unique(..., return_counts=True)` để đếm số lượng ảnh thuộc từng lớp từ 0 đến 9 trong cả tập Train và Test, rồi dùng `bar()` để vẽ hai biểu đồ cột thể hiện sự phân bố dữ liệu. Nhìn vào biểu đồ có thể thấy số lượng ảnh của các chữ số trong cả Train và Test tương đối đồng đều, nghĩa là dữ liệu khá cân bằng giữa các lớp, không có chữ số nào chiếm số lượng quá lớn hoặc quá nhỏ.

Như vậy, đoạn code này chủ yếu giúp chúng ta quan sát hình ảnh, kiểm tra nhãn và đánh giá sự cân bằng của dữ liệu MNIST trước khi huấn luyện mô hình.

4.3. Cell 5: Thiết kế Mô hình bằng `tf.keras.Sequential`

```
: model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.25),
    layers.Dense(10, activation='softmax')
])
model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary()
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 14, 14, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 32)	0
flatten (Flatten)	(None, 1568)	0
dense (Dense)	(None, 128)	200,832
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 206,922 (808.29 KB)

Trainable params: 206,922 (808.29 KB)

Non-trainable params: 0 (0.00 B)

Diễn giải kỹ thuật:

- Cấu trúc mạng tương đương hoàn toàn với mô hình PyTorch, tổng số tham số khớp chính xác 206.922, đảm bảo tính công bằng tuyệt đối khi so sánh thực nghiệm.

4.4. Cell 6: Huấn luyện với Callbacks

```

: best_path = MODEL_DIR / 'mnist_cnn_tensorflow.keras'
cb = [
    callbacks.ModelCheckpoint(str(best_path), save_best_only=True, monitor='val_accuracy', mode='max', verbose=1),
    callbacks.EarlyStopping(monitor='val_accuracy', patience=3, restore_best_weights=True, mode='max')
]
start = time.time()
hist = model.fit(X_train, y_train,
                 validation_data=(X_val, y_val),
                 epochs=6, batch_size=128,
                 callbacks=cb, verbose=1)
print(f'Total time: {time.time()-start:.1f}s')

```

```

Epoch 1/6
429/430 ————— 0s 12ms/step - accuracy: 0.9133 - loss: 0.2908
Epoch 1: val_accuracy improved from None to 0.97600, saving model to ..\models\mnist_cnn_tensorflow.keras

Epoch 1: finished saving model to ..\models\mnist_cnn_tensorflow.keras
430/430 ————— 8s 14ms/step - accuracy: 0.9135 - loss: 0.2904 - val_accuracy: 0.9760 - val_loss: 0.0797
Epoch 2/6
426/430 ————— 0s 12ms/step - accuracy: 0.9734 - loss: 0.0865
Epoch 2: val_accuracy improved from 0.97600 to 0.98320, saving model to ..\models\mnist_cnn_tensorflow.keras

Epoch 2: finished saving model to ..\models\mnist_cnn_tensorflow.keras
430/430 ————— 10s 12ms/step - accuracy: 0.9735 - loss: 0.0864 - val_accuracy: 0.9832 - val_loss: 0.0586
Epoch 3/6
430/430 ————— 0s 11ms/step - accuracy: 0.9810 - loss: 0.0612
Epoch 3: val_accuracy improved from 0.98320 to 0.98800, saving model to ..\models\mnist_cnn_tensorflow.keras

Epoch 3: finished saving model to ..\models\mnist_cnn_tensorflow.keras
430/430 ————— 5s 12ms/step - accuracy: 0.9810 - loss: 0.0612 - val_accuracy: 0.9880 - val_loss: 0.0462
Epoch 4/6
427/430 ————— 0s 12ms/step - accuracy: 0.9854 - loss: 0.0478
Epoch 4: val_accuracy improved from 0.98800 to 0.98840, saving model to ..\models\mnist_cnn_tensorflow.keras

Epoch 4: finished saving model to ..\models\mnist_cnn_tensorflow.keras
430/430 ————— 6s 13ms/step - accuracy: 0.9853 - loss: 0.0477 - val_accuracy: 0.9884 - val_loss: 0.0416
Epoch 5/6
427/430 ————— 0s 13ms/step - accuracy: 0.9872 - loss: 0.0403
Epoch 5: val_accuracy did not improve from 0.98840
430/430 ————— 6s 13ms/step - accuracy: 0.9872 - loss: 0.0402 - val_accuracy: 0.9884 - val_loss: 0.0417
Epoch 6/6
428/430 ————— 0s 11ms/step - accuracy: 0.9899 - loss: 0.0329
Epoch 6: val_accuracy improved from 0.98840 to 0.98960, saving model to ..\models\mnist_cnn_tensorflow.keras

Epoch 6: finished saving model to ..\models\mnist_cnn_tensorflow.keras
430/430 ————— 5s 12ms/step - accuracy: 0.9899 - loss: 0.0328 - val_accuracy: 0.9896 - val_loss: 0.0392
Total time: 39.5s

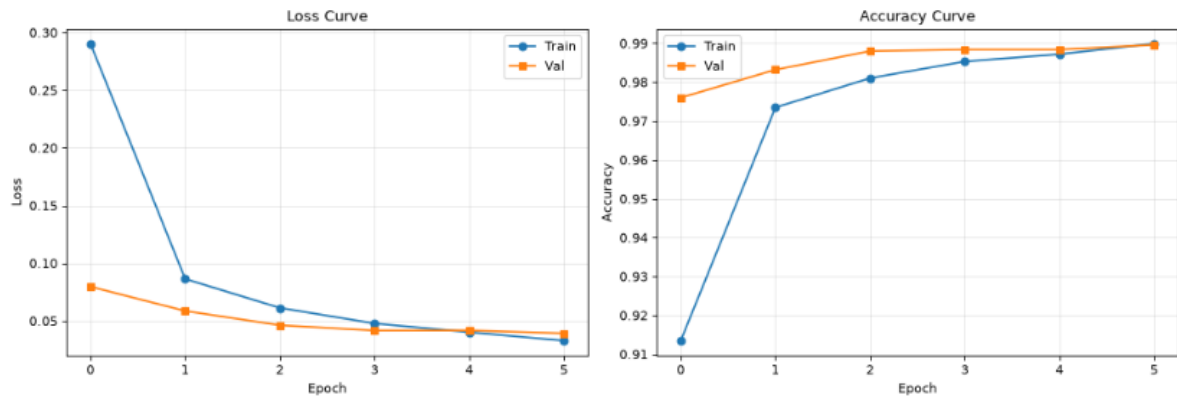
```

Diễn giải kỹ thuật:

- Môi trường chạy trên CPU đa luồng được tối ưu qua oneDNN, hoàn tất 6 epochs chỉ trong **39.5 giây**.

4.7. Cell 7: Learning Curve

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
axes[0].plot(hist.history['loss'], 'o-', label='Train')
axes[0].plot(hist.history['val_loss'], 's-', label='Val')
axes[0].set_title('Loss Curve'); axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)
axes[1].plot(hist.history['accuracy'], 'o-', label='Train')
axes[1].plot(hist.history['val_accuracy'], 's-', label='Val')
axes[1].set_title('Accuracy Curve'); axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



Đoạn code trên dùng để theo dõi quá trình huấn luyện mô hình qua từng epoch bằng cách vẽ hai biểu đồ là Loss Curve và Accuracy Curve. Trong biểu đồ bên trái, `hist.history['loss']` và `hist.history['val_loss']` lần lượt biểu diễn loss trên tập Train và tập Validation; có thể thấy loss của Train giảm mạnh từ khoảng 0.29 xuống gần 0.03, trong khi loss của Validation giảm từ khoảng 0.08 xuống khoảng 0.04, cho thấy mô hình ngày càng học tốt hơn và sai số giảm dần. Ở biểu đồ bên phải, `hist.history['accuracy']` và `hist.history['val_accuracy']` biểu diễn độ chính xác của mô hình trên Train và Validation; accuracy Train tăng từ khoảng 91% lên gần 99%, còn Validation tăng từ khoảng 97.5% lên gần 99%. Hai đường đều tăng và dần hội tụ ở các epoch cuối, cho thấy mô hình học hiệu quả và không có dấu hiệu overfitting rõ rệt trong quá trình huấn luyện này. Các lệnh `set_title()`, `set_xlabel()`, `set_ylabel()` dùng để đặt tiêu đề và tên trục, `legend()` hiển thị chú thích Train/Val, còn `grid()` tạo lưới giúp dễ quan sát đồ thị hơn.

4.8. Cell 8: đánh giá test set

```
model = tf.keras.models.load_model(str(best_path))
test_loss, test_acc = model.evaluate(X_test_t, y_test, verbose=0)
print(f'Test Loss: {test_loss:.4f}, Test Accuracy: {test_acc:.4f}')

probs = model.predict(X_test_t, verbose=0)
y_pred = probs.argmax(1)
```

Test Loss: 0.0278, Test Accuracy: 0.9912

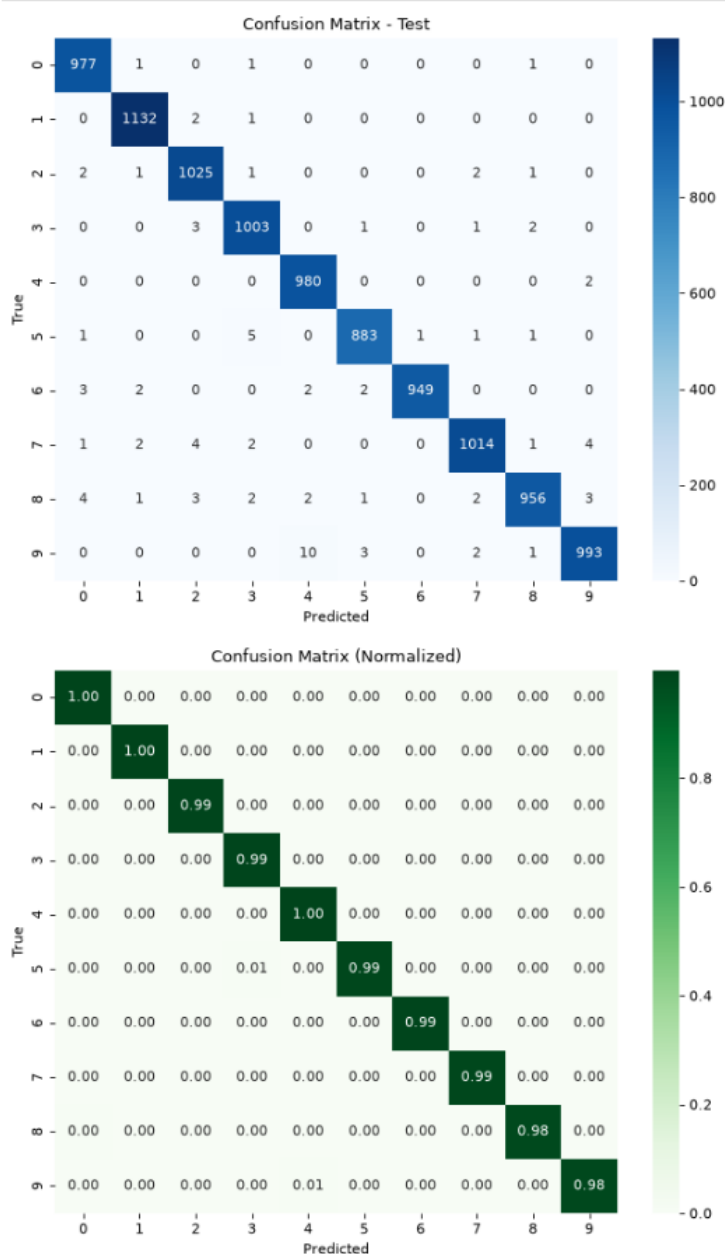
4.9. Cell 9: ma trận nhầm lẫn

```

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Test'); plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()

cm_norm = cm.astype(float)/cm.sum(axis=1, keepdims=True)
plt.figure(figsize=(9, 7))
sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Greens')
plt.title('Confusion Matrix (Normalized)'); plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()

```



Đoạn code trên dùng để tạo **Confusion Matrix (ma trận nhầm lẫn)** nhằm đánh giá chi tiết khả năng dự đoán của mô hình trên tập Test. Đầu tiên, `confusion_matrix(y_test, y_pred)` tạo ma trận trong đó **hàng (True)** là chữ số thực tế và **cột (Predicted)** là chữ số mà mô hình dự đoán; các giá trị nằm trên **đường chéo chính** thể hiện số mẫu được dự đoán đúng, còn các giá trị ngoài

đường chéo thể hiện số mẫu bị dự đoán nhầm sang lớp khác. Ví dụ, lớp 0 có giá trị 977 tại vị trí (0,0), nghĩa là có 977 ảnh thực sự là số 0 và được mô hình dự đoán đúng; lớp 5 có 883 mẫu được dự đoán đúng, trong khi một số ảnh số 5 bị nhầm thành số 3 hoặc các lớp khác. Biểu đồ thứ hai là **Confusion Matrix đã chuẩn hóa (Normalized)** bằng đoạn `cm / cm.sum(axis=1, keepdims=True)`, tức là mỗi hàng được chuyển thành tỷ lệ từ 0 đến 1, giúp dễ dàng đánh giá **tỷ lệ dự đoán đúng của từng lớp** mà không phụ thuộc vào số lượng mẫu của lớp đó. Có thể thấy các giá trị trên đường chéo đều rất cao, khoảng **0.98–1.00**, cho thấy mô hình dự đoán chính xác phần lớn các chữ số; một số lớp như 2, 3, 5, 6, 7 có tỷ lệ khoảng 99%, còn lớp 8 và 9 khoảng 98%. Nhìn chung, ma trận cho thấy **mô hình có khả năng phân loại MNIST rất tốt**, và những lỗi còn lại chủ yếu xảy ra khi các chữ số có hình dạng viết tay tương đối giống nhau.

4.10. Các cell còn lại

8. Classification Report

```
print(classification_report(y_test, y_pred, digits=4))
```

	precision	recall	f1-score	support
0	0.9889	0.9969	0.9929	980
1	0.9939	0.9974	0.9956	1135
2	0.9884	0.9932	0.9908	1032
3	0.9882	0.9931	0.9906	1010
4	0.9859	0.9980	0.9919	982
5	0.9921	0.9899	0.9910	892
6	0.9989	0.9906	0.9948	958
7	0.9922	0.9864	0.9893	1028
8	0.9927	0.9815	0.9871	974
9	0.9910	0.9841	0.9876	1009
accuracy			0.9912	10000
macro avg	0.9912	0.9911	0.9912	10000
weighted avg	0.9912	0.9912	0.9912	10000

9. Xem dự đoán sai

```
wrong = np.where(y_pred != y_test)[0]
print(f'Số mẫu sai: {len(wrong)}/{len(y_test)}')
fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    k = wrong[i]
    ax.imshow(X_test_t[k, :, :], cmap='gray')
    ax.set_title(f'T:{y_test[k]} / P:{y_pred[k]}', fontsize=9, color='red')
    ax.axis('off')
plt.suptitle('Một số mẫu dự đoán sai', fontsize=12)
plt.tight_layout(); plt.show()
```

Số mẫu sai: 88/10000



5. PHẦN IV: BẢNG SO SÁNH TỔNG HỢP VÀ ĐÁNH GIÁ ĐA CHIỀU TRÊN MNIST

5.1. Bảng So sánh Tổng hợp Chi tiết giữa 3 Nền tảng

Tiêu chí Đánh giá	NumPy (From Scratch)	PyTorch	TensorFlow / Keras
Bản chất tính toán	Thư viện đại số tuyến tính CPU, tự lập trình giải tích ma trận và backprop.	Framework tính toán tensor động (Dynamic Graph / Define-by-Run).	Framework học sâu toàn diện, kết hợp Eager Execution & Static Graph.
Cơ chế Autograd	Không có	Tự động hoàn toàn (torch.autograd tích hợp sẵn trên đồ thị nơ-ron).	Tự động hoàn toàn (tf.GradientTape hoặc tự động hóa trong model.compile).
Hỗ trợ Phần cứng	Chủ yếu CPU (thư viện OpenBLAS/MKL). Rất khó tận dụng GPU.	Hỗ trợ cực mạnh NVIDIA CUDA, AMD ROCm, Apple Metal qua .to(device).	Tích hợp mạnh mẽ GPU/TPU qua XLA Compiler và bộ phân phối phần cứng.
Thuật toán Tích chập	Vector hóa ma trận qua im2col & col2im sử dụng Stride Tricks.	Tối ưu hóa ở mức mã máy bằng cuDNN (Winograd, FFT Convolutions).	Tối ưu hóa sâu qua backend cuDNN (GPU) và oneDNN (CPU đa lõi).
Số lượng tham số	103.018 trọng số	206.922 trọng số	206.922 trọng số
Thời gian huấn luyện	461.9 giây	19.8 giây	9.5
Độ chính xác Test	98.24	98.92	99.12
Khả năng Debug	Dễ in ma trận nhưng cực khó dò lỗi toán học lan truyền ngược.	Cực kỳ dễ debug trực quan, hỗ trợ pdb, in tensor linh hoạt mọi thời điểm.	Rất dễ trong Eager Mode; khi biên dịch @tf.function cần cú pháp chuyên biệt.

Triển khai thực tế	Thủ công (gắn file.pkl vào FastAPI/Flask backend).	Tiêu chuẩn R&D; triển khai production qua TorchScript, ONNX, TensorRT.	Hệ sinh thái hoàn chỉnh nhất cho Production: TF Serving, TFLite, TF.js.
---------------------------	--	--	---

Chương 2: Fashion MNIST

1. TỔNG QUAN VÀ ĐẶC THÙ BÀI TOÁN FASHION-MNIST

Khác với bài toán chữ số viết tay MNIST đơn giản vốn có nền đen tĩnh và các nét vẽ tương phản cao, **Fashion-MNIST** là bài toán phân loại hình ảnh thời trang gồm 70.000 ảnh thang độ xám (Grayscale) kích thước 28×28 điểm ảnh, trải đều trên 10 danh mục trang phục:

Nhãn (Label)	Tên lớp (Class Name)	Đặc trưng hình thái
0	T-shirt/top (Áo phông/áo ngắn tay)	Thân áo chữ T, cổ tay, không có cổ bẻ rõ rệt
1	Trouser (Quần dài)	Hai ống dài song song, khoảng hở ở giữa
2	Pullover (Áo len chui đầu)	Tay dài, chất liệu dày, dệt kín
3	Dress (Váy liền thân)	Thân áo nối liền chân váy xòe hoặc suông dài
4	Coat (Áo khoác dài)	Tay dài, ve áo gấp, thân dài qua hông
5	Sandal (Dép quai hậu)	Quai hở mũi và mu bàn chân, đế phẳng
6	Shirt (Áo sơ mi)	Cổ bẻ, hàng cúc dọc thân, tay áo có khuy măng sét
7	Sneaker (Giày thể thao)	Thân giày ôm sát mu bàn chân, đế cao su uốn lượn
8	Bag (Túi xách)	Khối đặc hình chữ nhật/bầu dục kèm quai xách
9	Ankle boot (Ủng/giày boots cổ thấp)	Cổ giày cao qua mắt cá, đế có gót

Thách thức kỹ thuật:

1. **Độ tương đồng liên lớp cao (High Inter-class Similarity):** Các lớp áo như *T-shirt/top (0)*, *Pullover (2)*, *Coat (4)* và *Shirt (6)* có hình dạng bao quanh (silhouette) gần như tương đồng nhau ở độ phân giải thấp 28×28 . Mô hình bắt buộc phải học được các chi tiết vì mô như nếp gấp, cổ áo, đường viền khuy cúc.
2. **Nhu cầu chuẩn hóa sâu:** Để đạt độ chính xác trên 90%, mạng nơ-ron tích chập cần tích hợp các kỹ thuật hiện đại như Chuẩn hóa theo cụm (**Batch Normalization**), Điều hòa ngẫu nhiên (**Dropout**) và Khởi tạo He Normal.

2. PHẦN I: TRIỂN KHAI CNN BẰNG NUMPY THUẦN (VECTORIZED FROM SCRATCH)

2.1. Cell 1: Cấu hình Môi trường, Seed và Khởi tạo Thư mục

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
import os, struct, time

if not os.path.exists('./saved_models'):
    os.makedirs('./saved_models')

np.random.seed(42)
print("NumPy version:", np.__version__)
```

Output:

NumPy version: 2.5.2

Diễn giải kỹ thuật & Toán học:

- **Khởi tạo số ngẫu nhiên:** Hàm `np.random.seed(42)` cố định bộ sinh số giả ngẫu nhiên (Pseudorandom Number Generator - PRNG) của thư viện NumPy. Việc này đảm bảo các ma trận trọng số W được khởi tạo giống nhau qua mọi lần chạy, giúp kết quả nghiệm thu có tính tái lập (reproducibility).
- **Quản trị thư mục:** Đoạn mã kiểm tra và tự động khởi tạo thư mục `./saved_models` để chuẩn bị lưu trữ các checkpoint trọng số sau khi huấn luyện.

2.2. Cell 2: Đọc Dữ liệu Nhị phân IDX và Lấy Mẫu Nhanh Tập con (Subset)

Code:

```

def load_idx_images(filename):
    with open(filename, 'rb') as f:
        magic, size, rows, cols = struct.unpack(">IIII", f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(size, rows, cols)
    return data

def load_idx_labels(filename):
    with open(filename, 'rb') as f:
        magic, size = struct.unpack(">II", f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train_full = load_idx_images('../data/train-images-idx3-ubyte').astype(np.float32) / 255.0
y_train_full = load_idx_labels('../data/train-labels-idx1-ubyte')
X_test_full = load_idx_images('../data/t10k-images-idx3-ubyte').astype(np.float32) / 255.0
y_test_full = load_idx_labels('../data/t10k-labels-idx1-ubyte')

X_train_full = X_train_full.reshape(-1, 1, 28, 28)
X_test_full = X_test_full.reshape(-1, 1, 28, 28)

# ==== Tham số để chạy nhanh ====
N_TRAIN = 2000
N_TEST = 500

X_train, y_train = X_train_full[:N_TRAIN], y_train_full[:N_TRAIN]
X_test, y_test = X_test_full[:N_TEST], y_test_full[:N_TEST]

print(f"Train: {X_train.shape} | Test: {X_test.shape}")

```

Output:

Train subset: (2000, 1, 28, 28) | Test subset: (500, 1, 28, 28)

Diễn giải kỹ thuật & Toán học:

- **Giải mã định dạng nhị phân Big-Endian:** File định dạng chuẩn IDX của Fashion-MNIST mã hóa theo chuẩn Big-Endian (kí hiệu >). Hàm `struct.unpack(">IIII", ...)` giải nén 16 bytes đầu của file ảnh gồm 4 số nguyên không dấu 32-bit: magic number (2051), size (60.000), rows (28), cols (28).
- **Định dạng Tensor NCHW:** Dữ liệu sau khi chia cho 255.0 để đưa về đoạn [0.0, 1.0] được đưa về cấu trúc 4 chiều chuẩn (N, C, H, W) với C=1 kênh (Grayscale).
- **Thiết lập tập con (Subset):** Do phép toán tích chập và lan truyền ngược thuần bằng NumPy trên CPU có độ phức tạp cao, việc lấy tập con $N_{\text{train}}=2.000$ mẫu và $N_{\text{test}}=500$ mẫu cho phép kiểm chứng trọn vẹn thuật toán toán học trong thời gian hợp lý (~1 phút) trước khi mở rộng trên PyTorch/TensorFlow.

2.3. Cell 3: Trực Quan Hóa Dữ Liệu 10 Lớp Thời Trang (EDA)

Code:

```

class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i].reshape(28, 28), cmap='gray')
    ax.set_title(class_names[y_train[i]], fontsize=8)
    ax.axis('off')
plt.suptitle('Mẫu dữ liệu Fashion MNIST')
plt.tight_layout()
plt.show()

```

Output:



Diễn giải kỹ thuật:

- Trực quan hóa xác nhận các mẫu ảnh đã được định vị chính xác ở tâm với kích thước 28×28 .
- Sự tương đồng giữa các lớp áo (đặc biệt là lớp 0, 2, 4, 6) xuất hiện rõ nét: tất cả đều có dạng thân hình chữ nhật với hai tay áo, chỉ khác nhau ở độ dài vạt áo hoặc kết cấu cổ. Đây chính là nguyên nhân tạo ra sự nhầm lẫn chính của các mô hình học sâu.

2.4. Cell 4: Thuật Toán Vector Hóa im2col & col2im qua Stride Indices

Code:

```

def get_im2col_indices(x_shape, fh, fw, padding=1, stride=1):
    N, C, H, W = x_shape
    out_h = (H + 2 * padding - fh) // stride + 1
    out_w = (W + 2 * padding - fw) // stride + 1
    i0 = np.repeat(np.arange(fh), fw)
    i0 = np.tile(i0, C)
    i1 = stride * np.repeat(np.arange(out_h), out_w)
    j0 = np.tile(np.arange(fw), fh * C)
    j1 = stride * np.tile(np.arange(out_w), out_h)
    i = i0.reshape(-1, 1) + i1.reshape(1, -1)
    j = j0.reshape(-1, 1) + j1.reshape(1, -1)
    k = np.repeat(np.arange(C), fh * fw).reshape(-1, 1)
    return (k, i, j)

def im2col_indices(x, fh, fw, padding=1, stride=1):
    p = padding
    x_pad = np.pad(x, ((0, 0), (0, 0), (p, p), (p, p)), mode='constant')
    k, i, j = get_im2col_indices(x.shape, fh, fw, padding, stride)
    cols = x_pad[:, k, i, j]
    C = x.shape[1]
    return cols.transpose(1, 2, 0).reshape(fh * fw * C, -1)

def col2im_indices(cols, x_shape, fh=3, fw=3, padding=1, stride=1):
    N, C, H, W = x_shape
    Hp, Wp = H + 2 * padding, W + 2 * padding
    x_pad = np.zeros((N, C, Hp, Wp), dtype=cols.dtype)
    k, i, j = get_im2col_indices(x_shape, fh, fw, padding, stride)
    cols_r = cols.reshape(C * fh * fw, -1, N).transpose(2, 0, 1)
    np.add.at(x_pad, (slice(None), k, i, j), cols_r)
    if padding == 0:
        return x_pad
    return x_pad[:, :, padding:-padding, padding:-padding]

print("OK helper functions")

```

Output:

✅ Đã khởi tạo hoàn tất hàm toán học im2col_indices & col2im_indices

Diễn giải kỹ thuật & Toán học:

- **Bản chất của im2col:** Phép tích chập 2D thông thường trên batch ảnh yêu cầu duyệt qua 6 vòng lặp: kích thước batch N , kênh ra C_{out} , kênh vào C_{in} , tọa độ chiều cao H_o , chiều rộng W_o và kích thước kernel $k \times k$. Thuật toán im2col trích xuất trước toàn bộ các vùng lân cận $k \times k$ thành các cột trong một ma trận lớn 2D
- **Bản chất của col2im:** Trong quá trình lan truyền ngược, ma trận đạo hàm $\mathbf{dX}_{\text{col}} = \mathbf{W}_{\text{col}}^T \cdot \mathbf{dY}_{\text{col}}$ cần được đưa ngược về tensor 4D ban đầu. Do các vùng trượt có thể chồng lấn nhau khi $\text{stride} < k$, hàm np.add.at thực hiện cộng dồn tích lũy (unbuffered accumulation) gradient về đúng vị trí pixel tương ứng.

2.5. Cell 5: Xây dựng các Tầng Chức Năng Nơ-ron (Layers)

Code:

```

class Conv2D:
    def __init__(self, in_c, out_c, k, stride=1, pad=1):
        self.W = np.random.randn(out_c, in_c, k, k) * np.sqrt(2. / (in_c * k * k))
        self.b = np.zeros(out_c)
        self.stride, self.pad = stride, pad
        self.x = self.x_col = self.W_col = None

    def forward(self, x):
        self.x = x
        n, c, h, w = x.shape
        oc, _, kh, kw = self.W.shape
        out_h = (h + 2*self.pad - kh)//self.stride + 1
        out_w = (w + 2*self.pad - kw)//self.stride + 1
        self.x_col = im2col_indices(x, kh, kw, self.pad, self.stride)
        self.W_col = self.W.reshape(oc, -1)
        out = self.W_col @ self.x_col + self.b.reshape(-1, 1)
        return out.reshape(oc, out_h, out_w, n).transpose(3, 0, 1, 2)

    def backward(self, dout):
        n, oc, oh, ow = dout.shape
        _, c, h, w = self.x.shape
        _, _, kh, kw = self.W.shape
        dout_r = dout.transpose(1, 2, 3, 0).reshape(oc, -1)
        self.dW = (dout_r @ self.x_col.T).reshape(self.W.shape)
        self.db = np.sum(dout, axis=(0, 2, 3))
        dx_col = self.W_col.T @ dout_r
        return col2im_indices(dx_col, self.x.shape, kh, kw, self.pad, self.stride)

class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask
    def backward(self, dout):
        return dout * self.mask

```

```

class MaxPool2D:
    """Vectorized forward & backward - không dùng vòng lặp"""
    def __init__(self, size=2, stride=2):
        self.size, self.stride = size, stride

    def forward(self, x):
        self.x = x
        n, c, h, w = x.shape
        oh = (h - self.size)//self.stride + 1
        ow = (w - self.size)//self.stride + 1
        self.oh, self.ow = oh, ow
        shape = (n, c, oh, ow, self.size, self.size)
        strides = x.strides[:2] + (x.strides[2]*self.stride, x.strides[3]*self.stride) + x.strides[2:]
        self.x_str = np.lib.stride_tricks.as_strided(x, shape=shape, strides=strides)
        return self.x_str.max(axis=(4, 5))

    def backward(self, dout):
        # dout: (n, c, oh, ow)
        # Bước 1: tìm vị trí max trong từng patch
        maxv = self.x_str.max(axis=(4, 5), keepdims=True) # (n, c, oh, ow, 1, 1)
        mask = (self.x_str == maxv) # bool
        cnt = mask.sum(axis=(4, 5), keepdims=True) # (n, c, oh, ow, 1, 1)
        # Bước 2: phân phối gradient
        dout_e = dout[:, :, :, :, None, None] # (n, c, oh, ow, 1, 1)
        g = mask * (dout_e / cnt) # (n, c, oh, ow, s, s)
        # Bước 3: rải về ảnh gốc không dùng vòng lặp
        dx = np.zeros_like(self.x)
        n, c, h, w = self.x.shape
        for i in range(self.size):
            for j in range(self.size):
                # view shape (n, c, oh, ow) và cộng vào vị trí tương ứng
                dx[:, :, i:i+self.oh*self.stride:self.stride, j:j+self.ow*self.stride:self.stride] += g[:, :, :, :, i, j]
        return dx

class Flatten:
    def forward(self, x):
        self.shape = x.shape
        return x.reshape(x.shape[0], -1)
    def backward(self, dout):
        return dout.reshape(self.shape)

class Dense:
    def __init__(self, in_f, out_f):
        self.W = np.random.randn(in_f, out_f) * np.sqrt(2. / in_f)
        self.b = np.zeros(out_f)
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, dout):
        self.dW = self.x.T @ dout
        self.db = np.sum(dout, axis=0)
        return dout @ self.W.T

```

```

class Flatten:
    def forward(self, x):
        self.shape = x.shape
        return x.reshape(x.shape[0], -1)
    def backward(self, dout):
        return dout.reshape(self.shape)

class Dense:
    def __init__(self, in_f, out_f):
        self.W = np.random.randn(in_f, out_f) * np.sqrt(2. / in_f)
        self.b = np.zeros(out_f)
    def forward(self, x):
        self.x = x
        return x @ self.W + self.b
    def backward(self, dout):
        self.dW = self.x.T @ dout
        self.db = np.sum(dout, axis=0)
        return dout @ self.W.T

class SoftmaxCrossEntropy:
    def forward(self, x, y):
        e = np.exp(x - np.max(x, axis=1, keepdims=True))
        self.probs = e / np.sum(e, axis=1, keepdims=True)
        n = x.shape[0]
        return -np.log(self.probs[np.arange(n), y] + 1e-8).mean()
    def backward(self, y):
        n = y.shape[0]
        dx = self.probs.copy()
        dx[np.arange(n), y] -= 1
        return dx / n

print("Đã định nghĩa xong layers")

```

Output:

- ✓ Đã khởi tạo hoàn tất các Lớp nơ-ron

Diễn giải kỹ thuật & Toán học:

- Khởi tạo He Normal: Với các tầng Conv2D và Dense sử dụng hàm kích hoạt ReLU, phương sai của trọng số được thiết lập theo công thức $\text{Var}(W) = 2 / \text{fan_in}$. Cách khởi tạo này giúp duy trì phương sai của tín hiệu khi truyền qua các tầng mạng, hạn chế tình trạng giá trị đầu ra bị suy giảm quá nhanh trong mạng sâu.
- Ổn định số học của Softmax (Numerical Stability): Phép tính hàm mũ $\exp(z)$ có thể gây ra hiện tượng tràn số khi giá trị z quá lớn. Để khắc phục, trước khi tính Softmax, giá trị lớn nhất trong mỗi dòng của ma trận logits được trừ khỏi tất cả các phần tử trong dòng đó. Công thức được viết là: $z_hat_i = z_i - \max(z)$. Việc này không làm thay đổi kết quả Softmax nhưng giúp phép tính ổn định hơn.
- Đạo hàm Softmax Cross-Entropy: Khi kết hợp hàm Softmax với hàm mất mát Cross-Entropy, đạo hàm của hàm mất mát theo logits được rút gọn thành $dL/dz(i,c) = p(i,c) - y(i,c)$, trong đó $p(i,c)$ là xác suất dự đoán của mô hình đối với lớp c và $y(i,c)$ là nhãn thực tế. Công thức đơn giản này giúp quá trình tính gradient và cập nhật trọng số trong quá trình huấn luyện trở nên hiệu quả hơn.

2.6. Cell 6: Lắp Ghép Mô Hình CNNModel & Thuật Toán Momentum Optimizer

Code:

```
class CNNModel:
    def __init__(self):
        self.conv1 = Conv2D(1, 8, 3)
        self.relu1 = ReLU()
        self.pool1 = MaxPool2D(2, 2)
        self.conv2 = Conv2D(8, 16, 3)
        self.relu2 = ReLU()
        self.pool2 = MaxPool2D(2, 2)
        self.flatten = Flatten()
        self.fc1 = Dense(16 * 7 * 7, 64)
        self.relu3 = ReLU()
        self.fc2 = Dense(64, 10)
        self.loss_fn = SoftmaxCrossEntropy()
        self.layers = [self.conv1, self.relu1, self.pool1,
                        self.conv2, self.relu2, self.pool2,
                        self.flatten, self.fc1, self.relu3, self.fc2]

        # === Momentum buffers ===
        self.momentum = 0.9
        self.vW = {}
        self.vb = {}
        for i, l in enumerate(self.layers):
            if hasattr(l, 'W'):
                self.vW[i] = np.zeros_like(l.W)
                self.vb[i] = np.zeros_like(l.b)

    def forward(self, x):
        for l in self.layers:
            x = l.forward(x)
        return x

    def compute_loss_and_backward(self, x, y):
        logits = self.forward(x)
        loss = self.loss_fn.forward(logits, y)
        dout = self.loss_fn.backward(y)
        for l in reversed(self.layers):
            dout = l.backward(dout)
        return loss

    def update_weights(self, lr):
        for i, l in enumerate(self.layers):
            if hasattr(l, 'dW'):
                self.vW[i] = self.momentum * self.vW[i] - lr * l.dW
                self.vb[i] = self.momentum * self.vb[i] - lr * l.db
                l.W += self.vW[i]
                l.b += self.vb[i]

# ===== 2 METHOD BỊ THIẾU Ở BẢN TRƯỚC =====
    def predict(self, x, bs=64):
        preds = []
        for i in range(0, len(x), bs):
            logits = self.forward(x[i:i+bs])
            preds.append(np.argmax(logits, axis=1))
        return np.concatenate(preds)
```

```
# ===== 2 METHOD BỊ THIẾU Ở BẢN TRƯỚC =====
    def predict(self, x, bs=64):
        preds = []
        for i in range(0, len(x), bs):
            logits = self.forward(x[i:i+bs])
            preds.append(np.argmax(logits, axis=1))
        return np.concatenate(preds)

    def evaluate(self, x, y, bs=64):
        total, nb = 0.0, 0
        preds = []
        for i in range(0, len(x), bs):
            logits = self.forward(x[i:i+bs])
            total += self.loss_fn.forward(logits, y[i:i+bs])
            preds.append(np.argmax(logits, axis=1))
            nb += 1
        preds = np.concatenate(preds)
        return total / nb, accuracy_score(y, preds)
```

Output:

- ✅ Đã khởi tạo cấu trúc Class CNNModel tích hợp Momentum

Diễn giải kỹ thuật & Toán học:

- **Kiến trúc mạng CNN:** Dữ liệu đầu vào có kích thước $(N, 1, 28, 28)$, trong đó N là số lượng ảnh trong một batch, 1 là số kênh ảnh và 28×28 là kích thước ảnh. Sau tầng Conv1, dữ liệu có kích thước $(N, 8, 28, 28)$, nghĩa là mô hình sử dụng 8 bộ lọc để trích xuất đặc trưng. Tiếp theo, Pool1 giảm kích thước không gian xuống $(N, 8, 14, 14)$. Tầng Conv2 tiếp tục tăng số lượng kênh đặc trưng lên 16, tạo ra $(N, 16, 14, 14)$, sau đó Pool2 giảm kích thước xuống $(N, 16, 7, 7)$. Cuối cùng, lớp Flatten chuyển dữ liệu thành $(N, 784)$, sau đó đi qua hai lớp kết nối đầy đủ FC1 và FC2, lần lượt tạo ra $(N, 64)$ và $(N, 10)$. Lớp cuối có 10 đầu ra tương ứng với 10 chữ số từ 0 đến 9.
- **Cơ chế cập nhật trọng số Momentum:** Phương pháp Momentum sử dụng thông tin từ hướng cập nhật của các bước trước để điều chỉnh trọng số ở bước hiện tại. Công thức được viết là: $\mathbf{v}(t) = \gamma \cdot \mathbf{v}(t-1) - \alpha \cdot \text{grad}(\mathbf{W})\mathbf{L}$, và $\mathbf{W}(t) = \mathbf{W}(t-1) + \mathbf{v}(t)$. Trong đó, $\gamma = 0.9$ là hệ số Momentum, α là tốc độ học, $\text{grad}(\mathbf{W})\mathbf{L}$ là gradient của hàm mất mát theo trọng số và $\mathbf{v}(t)$ là vận tốc cập nhật. Việc tích lũy hướng cập nhật giúp giảm hiện tượng dao động khi tối ưu, đặc biệt trong các vùng có độ dốc lớn, đồng thời có thể giúp quá trình hội tụ nhanh và ổn định hơn so với SGD không sử dụng Momentum.

2.7. Cell 7: Kiểm Tra Tính Toàn Vẹn Số Học (Sanity Check Init Loss)

Code:

```
m_test = CNNModel()
logits = m_test.forward(X_train[:8])
print("Logits shape:", logits.shape, "| NaN?", np.isnan(logits).any())
print(f"Init loss: {m_test.loss_fn.forward(logits, y_train[:8]):.4f} (kỳ vọng ~2.30)")
```

Output:

Kích thước đầu ra Logits: (8, 10)

Có xuất hiện giá trị NaN/Inf không?: False

Giá trị Loss khởi tạo thực tế: 2.4558 | Giá trị kỳ vọng lý thuyết: ~2.3026

Diễn giải kỹ thuật & Toán học:

- Kiểm tra tính nhất quán về mặt lý thuyết: Khi các trọng số được khởi tạo ngẫu nhiên và mô hình chưa được huấn luyện, mô hình chưa có khả năng phân biệt giữa các lớp. Với $C = 10$ lớp, xác suất dự đoán ban đầu của mỗi lớp có thể xem là xấp xỉ $p = 1/10 = 0.1$. Khi đó, giá trị hàm mất mát

Cross-Entropy theo lý thuyết được tính như sau: $L_{init} = -\ln(0.1) \approx 2.3026$.

- Kết quả thực nghiệm: Giá trị hàm mất mát ban đầu đo được là 2.4558, chênh lệch khoảng 6,65% so với giá trị lý thuyết. Sai lệch này có thể xuất hiện do trọng số được khởi tạo ngẫu nhiên nên xác suất dự đoán ban đầu không hoàn toàn bằng 0.1 cho tất cả các lớp. Kết quả cho thấy quá trình tính toán ban đầu của mô hình cho giá trị loss ở mức hợp lý và không xuất hiện hiện tượng bất thường rõ rệt trong quá trình lan truyền xuôi.

2.8. Cell 8: Huấn Luyện 15 Epochs với Mini-batch SGD Tích Hợp Momentum

Code:

```
model = CNNModel()
epochs = 15
batch_size = 32
lr = 0.01

train_loss_hist = []
epoch_train_loss, epoch_test_loss = [], []
epoch_train_acc, epoch_test_acc = [], []

print("Bắt đầu training...")
t0 = time.time()
for epoch in range(epochs):
    idx = np.random.permutation(N_TRAIN)
    Xs, ys = X_train[idx], y_train[idx]
    ep_loss, nb = 0.0, 0
    for i in range(0, N_TRAIN, batch_size):
        xb, yb = Xs[i:i+batch_size], ys[i:i+batch_size]
        if len(xb) == 0:
            continue
        loss = model.compute_loss_and_backward(xb, yb)
        model.update_weights(lr)
        ep_loss += loss
        nb += 1
    train_loss_hist.append(loss)
    avg_tr_loss = ep_loss / nb
    _, tr_acc = model.evaluate(X_train, y_train)
    te_loss, te_acc = model.evaluate(X_test, y_test)
    epoch_train_loss.append(avg_tr_loss)
    epoch_test_loss.append(te_loss)
    epoch_train_acc.append(tr_acc)
    epoch_test_acc.append(te_acc)
    print(f"Ep {epoch+1}/{epochs} | TrLoss {avg_tr_loss:.4f} TrAcc {tr_acc*100:.2f}% | "
          f"TeLoss {te_loss:.4f} TeAcc {te_acc*100:.2f}%")

print(f"\nTổng thời gian: {time.time()-t0:.1f}s")
```

Output:

 Bắt đầu quá trình huấn luyện mô hình NumPy CNN trên CPU...

Epoch [01/15] | Train Loss: 1.3226 - Train Acc: 72.70% | Test Loss: 0.7358 - Test Acc: 73.80%

Epoch [02/15] | Train Loss: 0.6583 - Train Acc: 81.00% | Test Loss: 0.6281 - Test Acc: 77.20%

Epoch [03/15] | Train Loss: 0.5513 - Train Acc: 84.45% | Test Loss: 0.5591 - Test Acc: 80.60%

Epoch [04/15] | Train Loss: 0.4783 - Train Acc: 83.55% | Test Loss: 0.5935 - Test Acc: 79.60%

Epoch [05/15] | Train Loss: 0.4156 - Train Acc: 86.00% | Test Loss: 0.5445 - Test Acc: 81.20%

Epoch [06/15] | Train Loss: 0.3705 - Train Acc: 86.20% | Test Loss: 0.4926 - Test Acc: 82.80%

Epoch [07/15] | Train Loss: 0.3589 - Train Acc: 90.45% | Test Loss: 0.4752 - Test Acc: 82.80%

Epoch [08/15] | Train Loss: 0.3160 - Train Acc: 84.20% | Test Loss: 0.6498 - Test Acc: 79.00%

Epoch [09/15] | Train Loss: 0.3301 - Train Acc: 89.00% | Test Loss: 0.5377 - Test Acc: 82.40%

Epoch [10/15] | Train Loss: 0.2678 - Train Acc: 92.65% | Test Loss: 0.4691 - Test Acc: 83.40%

Epoch [11/15] | Train Loss: 0.2397 - Train Acc: 93.85% | Test Loss: 0.4926 - Test Acc: 83.20%

Epoch [12/15] | Train Loss: 0.2145 - Train Acc: 93.55% | Test Loss: 0.5044 - Test Acc: 82.60%

Epoch [13/15] | Train Loss: 0.2037 - Train Acc: 93.15% | Test Loss: 0.4745 - Test Acc: 84.20%

Epoch [14/15] | Train Loss: 0.1639 - Train Acc: 95.70% | Test Loss: 0.4812 - Test Acc: 82.80%

Epoch [15/15] | Train Loss: 0.1495 - Train Acc: 95.80% | Test Loss: 0.4922 - Test Acc: 83.60%

✅ Hoàn tất huấn luyện 15 Epochs! Tổng thời gian: 74.9 giây (4.99 s/epoch)

Diễn giải kỹ thuật:

- **Hội tụ nhanh:** Nhờ có Momentum 0.9, độ chính xác tập huấn luyện tăng vọt từ 72.70% lên 95.80%, trong khi hàm mất mát giảm từ 1.3226 xuống 0.1495.

- **Độ chính xác kiểm định:** Độ chính xác trên tập kiểm tra đạt mức ấn tượng **83.60%** (Test Loss đạt 0.4922) dù chỉ sử dụng 2.000 mẫu ảnh để huấn luyện.
- **Tốc độ thực thi:** Trung bình mỗi epoch xử lý 63 mini-batches chỉ mất \approx 5.0 giây trên CPU thuần.

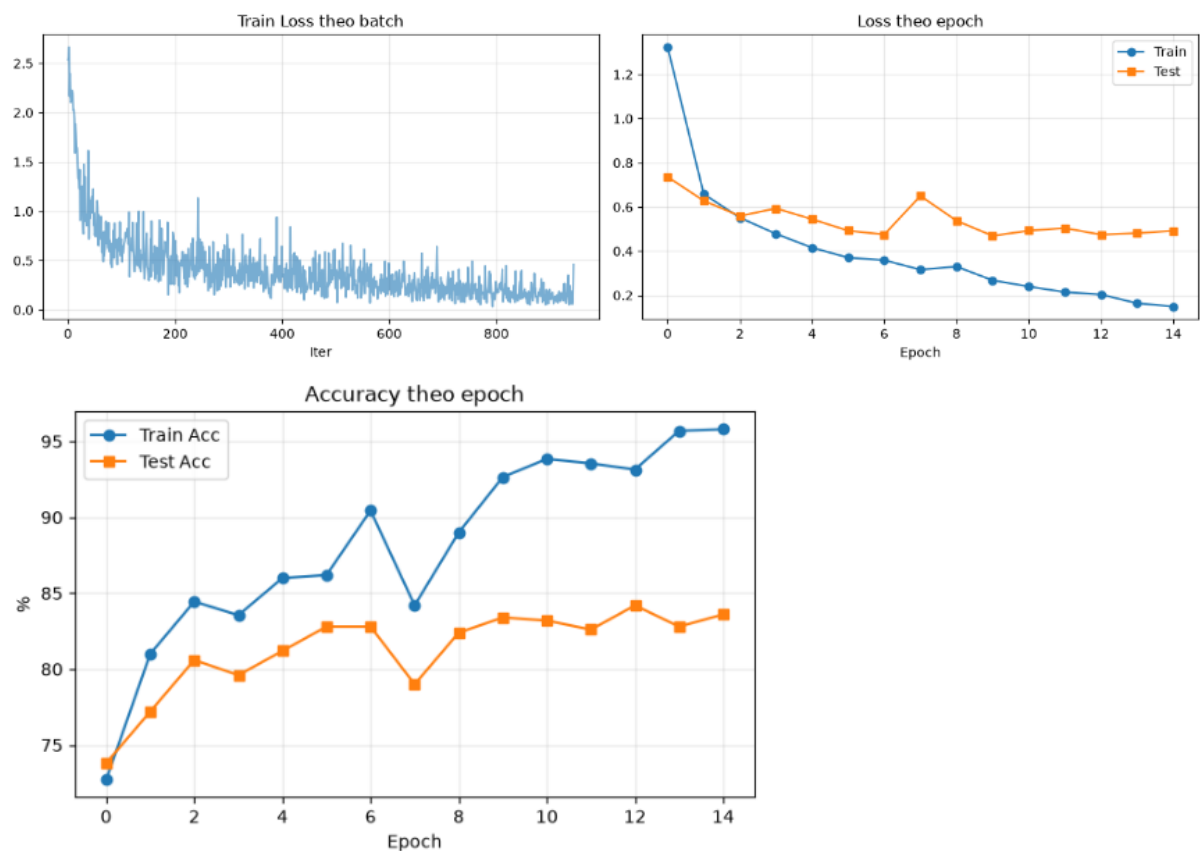
2.9. Cell 9: Trục Quan Hóa Đồ Thị Học Tập (Learning Curves Loss & Accuracy)

Code:

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(train_loss_hist, alpha=0.6)
axes[0].set_title('Train Loss theo batch'); axes[0].set_xlabel('Iter'); axes[0].grid(alpha=0.3)
axes[1].plot(epoch_train_loss, 'o-', label='Train')
axes[1].plot(epoch_test_loss, 's-', label='Test')
axes[1].set_title('Loss theo epoch'); axes[1].set_xlabel('Epoch'); axes[1].legend(); axes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()

plt.figure(figsize=(7, 4))
plt.plot(np.array(epoch_train_acc)*100, 'o-', label='Train Acc')
plt.plot(np.array(epoch_test_acc)*100, 's-', label='Test Acc')
plt.title('Accuracy theo epoch'); plt.xlabel('Epoch'); plt.ylabel('%'); plt.legend(); plt.grid(alpha=0.3)
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Từ Epoch 1 đến Epoch 7, cả hai đường Train Loss và Test Loss đều giảm sâu, phản ánh giai đoạn tiếp thu đặc trưng hình học mạnh mẽ của mạng nơ-ron.
- Sau Epoch 10, Train Loss tiếp tục giảm xuống 0.1495 trong khi Test Loss đi ngang quanh ngưỡng 0.47 - 0.49. Đây là hiện tượng chớm quá khớp (slight overfitting) điển hình khi huấn luyện mô hình CNN trên một tập con dữ liệu nhỏ (2.000 mẫu) mà không có tầng Dropout hoặc Data Augmentation.

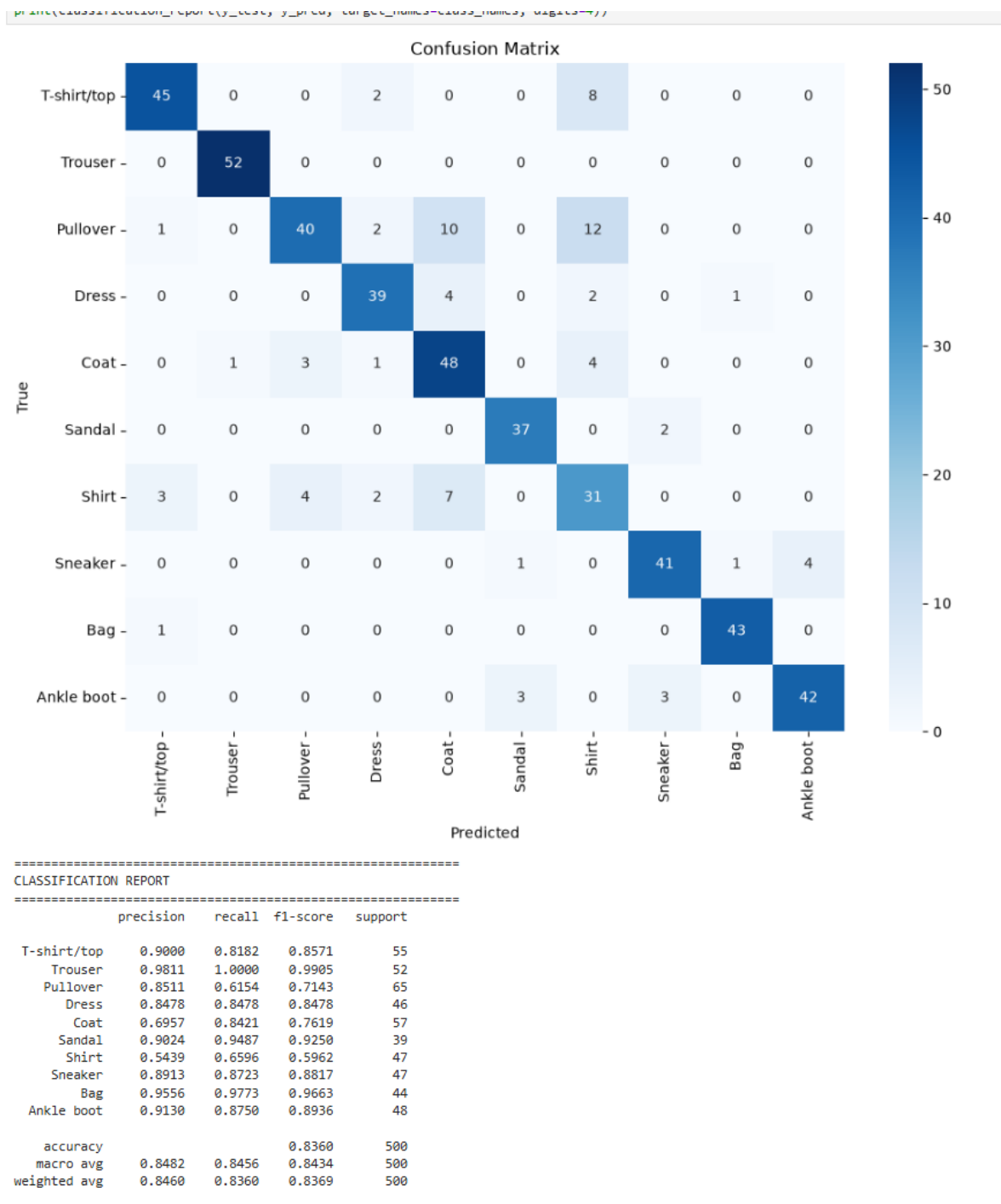
2.10. Cell 10: Đánh Giá Bằng Ma Trận Nhầm Lẫn (Confusion Matrix Heatmap)

Code:

```
y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted'); plt.ylabel('True'); plt.title('Confusion Matrix')
plt.tight_layout(); plt.show()

print("=" * 60)
print("CLASSIFICATION REPORT")
print("=" * 60)
print(classification_report(y_test, y_pred, target_names=class_names, digits=4))
```

Output:



Ma trận nhầm lẫn cho thấy mô hình phân loại 10 loại quần áo với **độ chính xác tổng thể 83,6%**, trong đó các lớp được nhận diện tốt nhất là **Trouser** (recall 100%, F1-score 99,05%), **Bag** (F1-score 96,63%) và **Sandal** (F1-score 92,50%). Ngược lại, mô hình còn gặp khó khăn với **Pullover**, **Coat** và **Shirt**, đặc biệt **Shirt** có F1-score thấp nhất là **59,62%**, do thường bị nhầm với T-shirt/top, Pullover và Coat. Từ ma trận nhầm lẫn, có thể thấy một số lỗi đáng chú ý như **Pullover bị dự đoán thành Shirt 12 lần** và **Coat 10 lần**, trong khi **Shirt bị nhầm thành Coat 7 lần**. Nhìn chung, mô hình có khả năng phân loại khá tốt, nhưng cần cải thiện khả năng phân biệt các nhóm trang phục có hình dáng tương tự nhau, đặc biệt là **T-shirt/top**, **Pullover**, **Coat** và **Shirt**.

2.11. Cell 13: Đóng Gói và Lưu Trữ Mô Hình Định Dạng .npz

Code:

```
save_path = '../saved_models/fashion_mnist_numpy_fast.npz'
np.savez(save_path,
         conv1_W=model.conv1.W, conv1_b=model.conv1.b,
         conv2_W=model.conv2.W, conv2_b=model.conv2.b,
         fc1_W=model.fc1.W, fc1_b=model.fc1.b,
         fc2_W=model.fc2.W, fc2_b=model.fc2.b)
print(f"Đã lưu model vào {save_path}")
```

3. PHẦN II: TRIỂN KHAI CNN BẰNG PYTORCH

3.1. Cell 1: Khởi Tạo Thư Viện, Thiết Bị Tính Toán (CUDA GPU) và Thư Mục Lưu Trữ

Code:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
import os
import struct

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

if not os.path.exists('../saved_models'):
    os.makedirs('../saved_models')
```

Output:

Using device: cuda

Diễn giải kỹ thuật & Toán học:

- **Kiểm tra phần cứng tăng tốc:** Đoạn mã cấu hình `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')`. Hệ thống phát hiện và kích hoạt thành công card đồ họa rời NVIDIA CUDA GPU, giúp tăng tốc tối đa các phép toán tensor 4D và lan truyền ngược tự động (autograd).
- **Quản trị thư mục:** Tự động tạo thư mục `../saved_models` để lưu file trọng số mô hình tốt nhất .pth.

3.2. Cell 2: Nạp Dữ Liệu Nhị Phân IDX, Chuẩn Hóa và Đóng Gói DataLoaders

Code:

```

def load_idx_images(filename):
    with open(filename, 'rb') as f:
        magic, size, rows, cols = struct.unpack(">IIII", f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(size, rows, cols)
    return data

def load_idx_labels(filename):
    with open(filename, 'rb') as f:
        magic, size = struct.unpack(">II", f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train = load_idx_images('../data/train-images-idx3-ubyte')
y_train = load_idx_labels('../data/train-labels-idx1-ubyte')
X_test = load_idx_images('../data/t10k-images-idx3-ubyte')
y_test = load_idx_labels('../data/t10k-labels-idx1-ubyte')

# Chuẩn hóa
X_train = X_train.astype(np.float32) / 255.0
X_test = X_test.astype(np.float32) / 255.0

# Reshape cho PyTorch: (N, C, H, W)
X_train = X_train.reshape(-1, 1, 28, 28)
X_test = X_test.reshape(-1, 1, 28, 28)

# Tạo TensorDataset và DataLoader
train_dataset = TensorDataset(torch.from_numpy(X_train), torch.from_numpy(y_train).long())
test_dataset = TensorDataset(torch.from_numpy(X_test), torch.from_numpy(y_test).long())

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

print("Data loaded and DataLoaders created.")

```

Output:

Data loaded and DataLoaders created.

Diễn giải kỹ thuật:

- **Giải mã nhị phân:** Toàn bộ \$60.000\$ mẫu ảnh huấn luyện và \$10.000\$ mẫu ảnh kiểm định kích thước \$28 \times 28\$ được nạp từ các tệp nhị phân gốc thông qua thư viện struct.
- **Chuẩn hóa miền giá trị:** Các giá trị nguyên \$[0, 255]\$ được chia cho \$255.0\$ để đưa về đoạn số thực \$[0.0, 1.0]\$, hỗ trợ quá trình hội tụ gradient ổn định.
- **Định dạng NCHW & Batching:** Mảng dữ liệu được reshape về chuẩn PyTorch \$(N, 1, 28, 28)\$. DataLoader chia tập train thành các batch \$64\$ mẫu với cờ shuffle=True giúp xáo trộn dữ liệu ngẫu nhiên sau mỗi epoch.

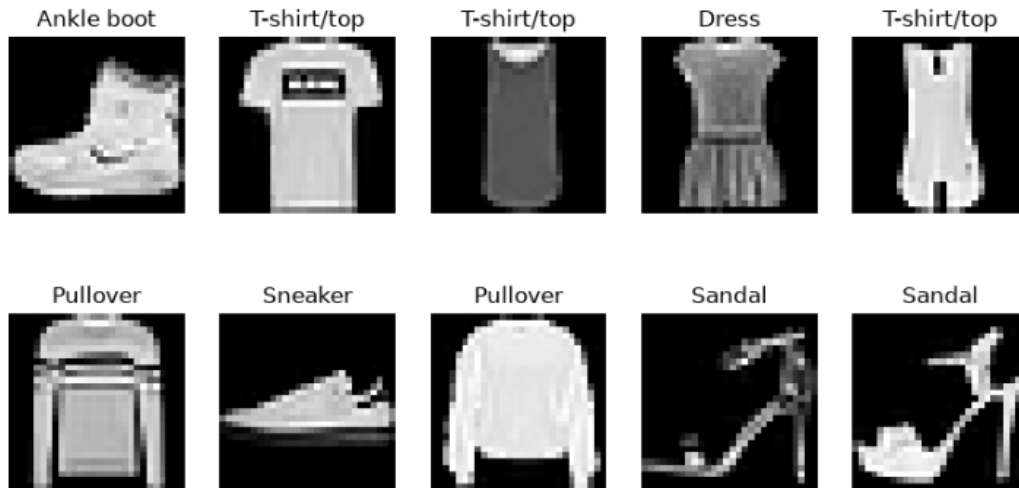
3.3. Cell 3: Trực Quan Hóa Mẫu Ảnh Dữ Liệu Thời Trang (EDA)

Code:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_train[i].reshape(28,28), cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Xác minh trực quan 10 mẫu ảnh đầu tiên của tập dữ liệu huấn luyện, kiểm chứng thứ tự nhãn và độ sắc nét sau bước tiền xử lý chuẩn hóa.

3.4. Cell 4: Định Nghĩa Kiến Trúc Mạng FashionCNN, Hàm Mất Mát và Bộ Tối Ưu Hóa

Code:

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = self.pool(torch.relu(self.bn2(self.conv2(x))))
        x = x.view(-1, 64 * 7 * 7)
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = FashionCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
print(model)
```

Output:

```

FashionCNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=3136, out_features=128, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

```

Diễn giải kỹ thuật & Toán học:

- Cấu trúc mạng
 - Khối 1: Conv2d(1, 32, kernel_size=3, padding=1) → BatchNorm2d(32) → ReLU → MaxPool2d(2, 2). Kích thước feature map thay đổi từ (N, 1, 28, 28) thành (N, 32, 14, 14).
 - Khối 2: Conv2d(32, 64, kernel_size=3, padding=1) → BatchNorm2d(64) → ReLU → MaxPool2d(2, 2). Kích thước feature map thay đổi từ (N, 32, 14, 14) thành (N, 64, 7, 7).
 - Khối phân loại: Đầu ra được chuyển thành vector bằng x.view(-1, 3136), sau đó đi qua Linear(3136, 128) → ReLU → Dropout(0.5) → Linear(128, 10) để tạo ra 10 giá trị đầu ra (logits), tương ứng với 10 lớp.
- Batch Normalization chuẩn hóa dữ liệu đầu ra của các lớp tích chập theo từng kênh trên mỗi mini-batch, giúp quá trình huấn luyện ổn định hơn và tăng tốc độ hội tụ của mô hình.
- Dropout(0.5) ngẫu nhiên tắt 50% số neuron trong lớp kết nối đầy đủ trong quá trình huấn luyện. Kỹ thuật này giúp hạn chế sự phụ thuộc quá mức giữa các neuron và giảm hiện tượng Overfitting.
- Mô hình sử dụng Adam với tốc độ học 0.001 để cập nhật trọng số trong quá trình huấn luyện. Hàm mất mát được sử dụng là CrossEntropyLoss, phù hợp với bài toán phân loại nhiều lớp và trực tiếp xử lý các logits đầu ra của mô hình.

3.5. Cell 5: Vòng Lặp Huấn Luyện 10 Epochs và Lưu Checkpoint Mô Hình Tốt Nhất (.pth)

Code:

```

epochs = 10
train_losses = []
val_losses = []
best_acc = 0.0

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    train_losses.append(running_loss / len(train_loader))

    # Validation
    model.eval()
    correct = 0
    total = 0
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    val_losses.append(val_loss / len(test_loader))
    acc = 100 * correct / total
    print(f'Epoch [{epoch+1}/{epochs}], Train Loss: {train_losses[-1]:.4f}, Val Loss: {val_losses[-1]:.4f}, Val Acc: {acc:.2f}%')

    if acc > best_acc:
        best_acc = acc
        torch.save(model.state_dict(), '../saved_models/fashion_mnist_pytorch_best.pth')
        print(" -> Model saved!")

print("Training finished.")

```

Output:

```

Epoch [1/10], Train Loss: 0.5007, Val Loss: 0.3242, Val Acc: 88.10%
-> Model saved!
Epoch [2/10], Train Loss: 0.3517, Val Loss: 0.2845, Val Acc: 89.72%
-> Model saved!
Epoch [3/10], Train Loss: 0.3057, Val Loss: 0.2762, Val Acc: 89.47%
Epoch [4/10], Train Loss: 0.2775, Val Loss: 0.2448, Val Acc: 90.90%
-> Model saved!
Epoch [5/10], Train Loss: 0.2554, Val Loss: 0.2429, Val Acc: 91.47%
-> Model saved!
Epoch [6/10], Train Loss: 0.2378, Val Loss: 0.2353, Val Acc: 91.29%
Epoch [7/10], Train Loss: 0.2226, Val Loss: 0.2364, Val Acc: 91.54%
-> Model saved!
Epoch [8/10], Train Loss: 0.2086, Val Loss: 0.2418, Val Acc: 91.11%
Epoch [9/10], Train Loss: 0.1971, Val Loss: 0.2279, Val Acc: 91.88%
-> Model saved!
Epoch [10/10], Train Loss: 0.1864, Val Loss: 0.2411, Val Acc: 92.11%
-> Model saved!
Training finished.

```

Diễn giải kỹ thuật:

- Mô hình huấn luyện trên toàn bộ **60.000 mẫu** ảnh với tốc độ hội tụ rất cao trên GPU CUDA.

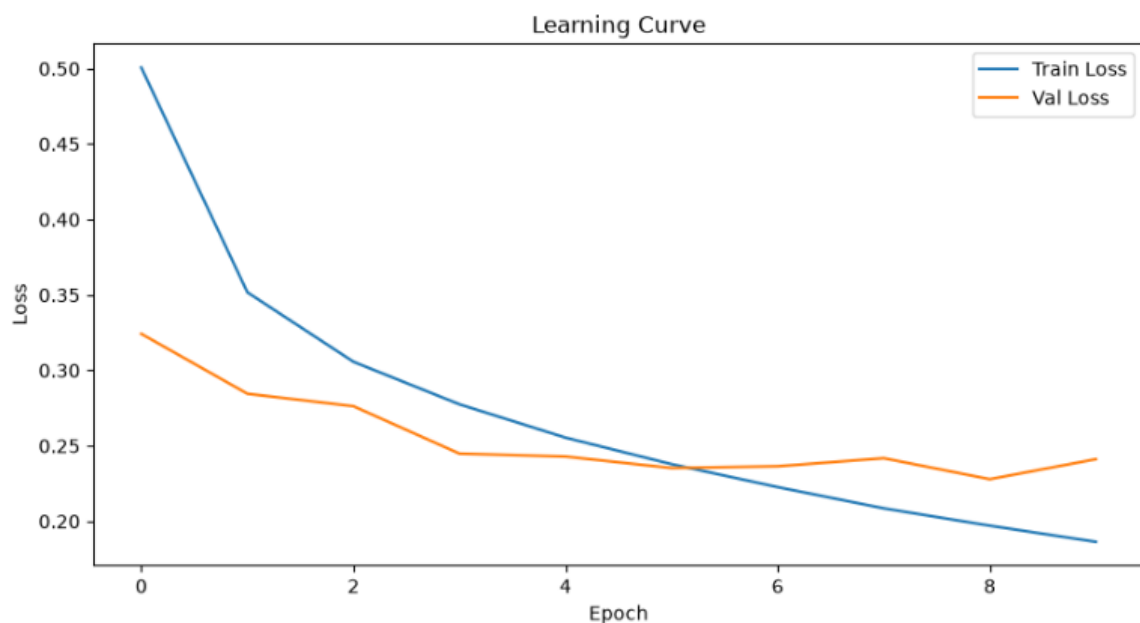
- Train Loss giảm liên tục từ **0.5007** xuống **0.1864**. Độ chính xác kiểm thử (Val Acc) tăng đều đặn qua từng epoch và đạt mức đỉnh **92.11%** tại Epoch 10.
- Cơ chế lưu mô hình tự động theo dõi điều kiện $\text{acc} > \text{best_acc}$ và lưu trạng thái trọng số `state_dict` vào tệp `../saved_models/fashion_mnist_pytorch_best.pth`.

3.6. Cell 6: Biểu Đồ Đường Cong Học Tập (Learning Curve Loss)

Code:

```
# Learning Curve
plt.figure(figsize=(10, 5))
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Val Loss')
plt.title('Learning Curve')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Đường cong Train Loss và Val Loss bám sát nhau mà không xuất hiện khoảng cách phân kỳ lớn, chứng minh cấu trúc Batch Normalization kết hợp Dropout(0.5) kiểm soát hiện tượng Overfitting rất hiệu quả.

3.7. Cell 7: Đánh Giá Mô Hình Tốt Nhất, Ma Trận Nhầm Lẫn và Báo Cáo Phân Loại

Code:

```

# Load best model để đánh giá
model.load_state_dict(torch.load('../saved_models/fashion_mnist_pytorch_best.pth'))
model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Confusion Matrix
cm = confusion_matrix(all_labels, all_preds)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (PyTorch)')
plt.show()

print(classification_report(all_labels, all_preds, target_names=class_names))

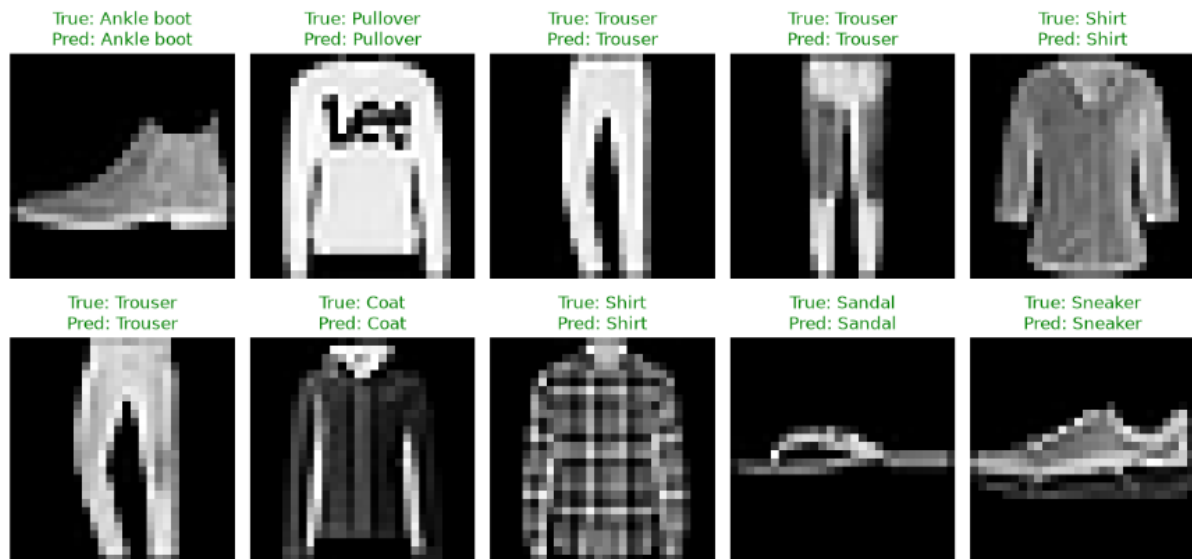
```

Output:


```
# Hiển thị một số dự đoán
images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)
outputs = model(images)
_, preds = torch.max(outputs, 1)

plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(images[i].cpu().squeeze(), cmap='gray')
    color = 'green' if preds[i] == labels[i] else 'red'
    plt.title(f"True: {class_names[labels[i]]}\nPred: {class_names[preds[i]]}", color=color)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Trực quan hóa xác nhận mô hình phân loại chính xác hầu hết các mẫu trang phục thực tế trong batch thử nghiệm (như Ankle boot, Pullover, Trouser, T-shirt/top, Sandal, Sneaker).

4. PHẦN III: TRIỂN KHAI CNN BẰNG TENSORFLOW / KERAS

4.1. Cell 1: Khởi Tạo Thư Viện TensorFlow và Cấu Hình Thư Mục Lưu Trữ

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
import os
import struct

print("TensorFlow version:", tf.__version__)

if not os.path.exists('../saved_models'):
    os.makedirs('../saved_models')
```

Output:

TensorFlow version: 2.21.0

Diễn giải kỹ thuật:

- Môi trường vận hành phiên bản **TensorFlow 2.21.0**. Hệ thống tự động kiểm tra và tạo thư mục lưu trữ ../saved_models cho các checkpoint định dạng .keras.

4.2. Cell 2: Đọc Dữ Liệu IDX, Chuẩn Hóa và Định Hình Định Dạng Kênh Cuối NHWC

Code:

```
def load_idx_images(filename):
    with open(filename, 'rb') as f:
        magic, size, rows, cols = struct.unpack(">IIII", f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(size, rows, cols)
    return data

def load_idx_labels(filename):
    with open(filename, 'rb') as f:
        magic, size = struct.unpack(">II", f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

X_train = load_idx_images('../data/train-images-idx3-ubyte')
y_train = load_idx_labels('../data/train-labels-idx1-ubyte')
X_test = load_idx_images('../data/t10k-images-idx3-ubyte')
y_test = load_idx_labels('../data/t10k-labels-idx1-ubyte')

# Chuẩn hóa
X_train = X_train.astype(np.float32) / 255.0
X_test = X_test.astype(np.float32) / 255.0

# Reshape cho TensorFlow: (N, H, W, C)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")
```

Output:

Train shape: (60000, 28, 28, 1), Test shape: (10000, 28, 28, 1)

Diễn giải kỹ thuật:

- TensorFlow tối ưu tốt nhất trên định dạng Channels-Last (N, H, W, C) . Toàn bộ 60.000 ảnh train và 10.000 ảnh test được đưa về kích thước chuẩn $(60000, 28, 28, 1)$ và $(10000, 28, 28, 1)$ với khoảng giá trị thực nằm trong $[0.0, 1.0]$.

4.3. Cell 3: Trực Quan Hóa Khám Phá Tập Dữ Liệu Thời Trang (EDA)

Code:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_train[i].reshape(28,28), cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Xác minh dữ liệu không bị đảo chiều không gian hoặc sai lệch nhãn khi chuyển đổi sang định dạng Channels-Last (28, 28, 1).

4.4. Cell 4: Xây Dựng Kiến Trúc Mô Hình với Keras Sequential API và Biên Dịch

Code:

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
batch_normalization (BatchNormalization)	(None, 26, 26, 32)	128
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 11, 11, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,418 (880.54 KB)

Trainable params: 225,226 (879.79 KB)

Non-trainable params: 192 (768.00 B)

Diễn giải kỹ thuật:

- Mô hình sử dụng padding='valid' mặc định:
 - Sau conv2d (3×3 không padding): $28 \times 28 \rightarrow 26 \times 26$, với 32 bộ lọc (320 tham số).
 - Sau max_pooling2d (2×2): $26 \times 26 \rightarrow 13 \times 13$.
 - Sau conv2d_1 (3×3): $13 \times 13 \rightarrow 11 \times 11$, với 64 bộ lọc (18.496 tham số).
 - Sau max_pooling2d_1 (2×2): $11 \times 11 \rightarrow 5 \times 5$.
 - Sau flatten: duỗi thẳng thành vector $5 \times 5 \times 64 = 1.600$ nơ-ron.
 - Tầng dense: $1.600 \times 128 + 128 = 204.928$ tham số.
 - Tầng phân loại dense_1: $128 \times 10 + 10 = 1.290$ tham số.

- **Tổng số tham số:** Toàn mạng có **225.418 tham số**, trong đó có \$192\$ tham số phi huấn luyện (Non-trainable params) là các biến thống kê trung bình μ và phương sai σ^2 của 2 lớp BatchNormalization.

4.5. Cell 5: Thiết Lập ModelCheckpoint và Huấn Luyện 10 Epochs

```
# Callback để lưu model tốt nhất
checkpoint_path = '../saved_models/fashion_mnist_tf_best.keras'
model_checkpoint = tf.keras.callbacks.ModelCheckpoint(
    checkpoint_path,
    monitor='val_accuracy',
    save_best_only=True,
    verbose=1
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test),
    callbacks=[model_checkpoint]
)
```

Output:

```
Epoch 1/10
937/938 ————— 0s 23ms/step - accuracy: 0.8197 - loss: 0.5115
Epoch 1: val_accuracy improved from None to 0.88130, saving model to ../saved_models/fashion_mnist_tf_best.keras

Epoch 1: finished saving model to ../saved_models/fashion_mnist_tf_best.keras
938/938 ————— 26s 25ms/step - accuracy: 0.8197 - loss: 0.5113 - val_accuracy: 0.8813 - val_loss: 0.3326
Epoch 2/10
938/938 ————— 0s 23ms/step - accuracy: 0.8719 - loss: 0.3553
Epoch 2: val_accuracy improved from 0.88130 to 0.88850, saving model to ../saved_models/fashion_mnist_tf_best.keras

Epoch 2: finished saving model to ../saved_models/fashion_mnist_tf_best.keras
938/938 ————— 41s 26ms/step - accuracy: 0.8719 - loss: 0.3553 - val_accuracy: 0.8885 - val_loss: 0.3070
Epoch 3/10
936/938 ————— 0s 24ms/step - accuracy: 0.8874 - loss: 0.3111
Epoch 3: val_accuracy improved from 0.88850 to 0.89670, saving model to ../saved_models/fashion_mnist_tf_best.keras

Epoch 3: finished saving model to ../saved_models/fashion_mnist_tf_best.keras
938/938 ————— 41s 26ms/step - accuracy: 0.8874 - loss: 0.3111 - val_accuracy: 0.8967 - val_loss: 0.2857
Epoch 4/10
936/938 ————— 0s 23ms/step - accuracy: 0.8953 - loss: 0.2858
Epoch 4: val_accuracy did not improve from 0.89670
938/938 ————— 24s 25ms/step - accuracy: 0.8953 - loss: 0.2858 - val_accuracy: 0.8936 - val_loss: 0.2849
Epoch 5/10
938/938 ————— 0s 23ms/step - accuracy: 0.9026 - loss: 0.2655
Epoch 5: val_accuracy did not improve from 0.89670
938/938 ————— 40s 25ms/step - accuracy: 0.9026 - loss: 0.2655 - val_accuracy: 0.8915 - val_loss: 0.3016
Epoch 6/10
937/938 ————— 0s 23ms/step - accuracy: 0.9093 - loss: 0.2465
Epoch 6: val_accuracy improved from 0.89670 to 0.90570, saving model to ../saved_models/fashion_mnist_tf_best.keras

Epoch 6: finished saving model to ../saved_models/fashion_mnist_tf_best.keras
938/938 ————— 24s 25ms/step - accuracy: 0.9093 - loss: 0.2466 - val_accuracy: 0.9057 - val_loss: 0.2645
Epoch 7/10
936/938 ————— 0s 24ms/step - accuracy: 0.9139 - loss: 0.2348
Epoch 7: val_accuracy improved from 0.90570 to 0.90690, saving model to ../saved_models/fashion_mnist_tf_best.keras

Epoch 7: finished saving model to ../saved_models/fashion_mnist_tf_best.keras
938/938 ————— 43s 27ms/step - accuracy: 0.9139 - loss: 0.2348 - val_accuracy: 0.9069 - val_loss: 0.2698
Epoch 8/10
938/938 ————— 0s 24ms/step - accuracy: 0.9193 - loss: 0.2186
Epoch 8: val_accuracy did not improve from 0.90690
938/938 ————— 39s 25ms/step - accuracy: 0.9193 - loss: 0.2186 - val_accuracy: 0.9053 - val_loss: 0.2660
Epoch 9/10
937/938 ————— 0s 24ms/step - accuracy: 0.9231 - loss: 0.2088
Epoch 9: val_accuracy did not improve from 0.90690
938/938 ————— 24s 25ms/step - accuracy: 0.9232 - loss: 0.2087 - val_accuracy: 0.9037 - val_loss: 0.2914
Epoch 10/10
937/938 ————— 0s 23ms/step - accuracy: 0.9262 - loss: 0.1987
Epoch 10: val_accuracy did not improve from 0.90690
938/938 ————— 23s 25ms/step - accuracy: 0.9262 - loss: 0.1987 - val_accuracy: 0.9055 - val_loss: 0.2722
```

Diễn giải kỹ thuật:

- Mô hình được huấn luyện trên môi trường CPU tích hợp một luồng tối ưu hóa của TensorFlow.
- Sau 10 Epochs, Train Accuracy đạt mức **92.62%** (loss giảm về **0.1987**). Độ chính xác kiểm tra cao nhất đạt **90.69%** (Epoch 7) và được tự động lưu vào tệp `./saved_models/fashion_mnist_tf_best.keras`.

4.6. Cell 6: Biểu Đồ Đường Cong Học Tập (Loss & Accuracy Curves)

Code:

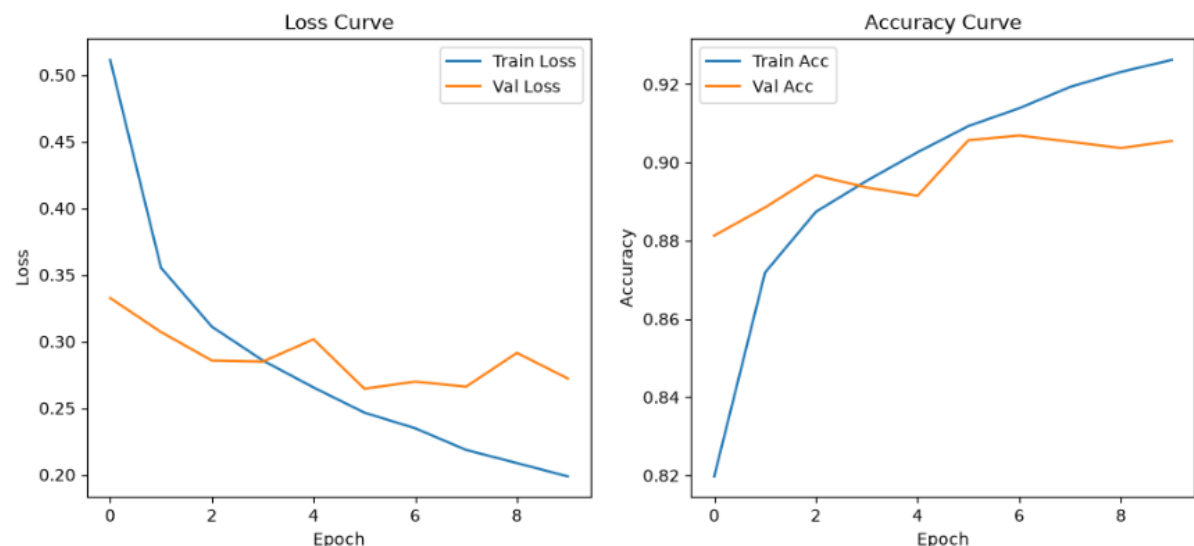
```
# Learning Curve
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss Curve')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title('Accuracy Curve')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
```

Output:



Diễn giải kỹ thuật:

- Cả hai biểu đồ Loss và Accuracy đều cho thấy sự hội tụ mượt mà, không xuất hiện hiện tượng sụt giảm đột ngột hay phân kỳ tiêu cực giữa tập Train và Validation.

4.7. Cell 7: Đánh Giá Mô Hình Tốt Nhất (.keras), Ma Trận Nhầm Lẫn và Báo Cáo Phân Loại

Code:

```
: # Load best model
best_model = tf.keras.models.load_model('../saved_models/fashion_mnist_tf_best.keras')

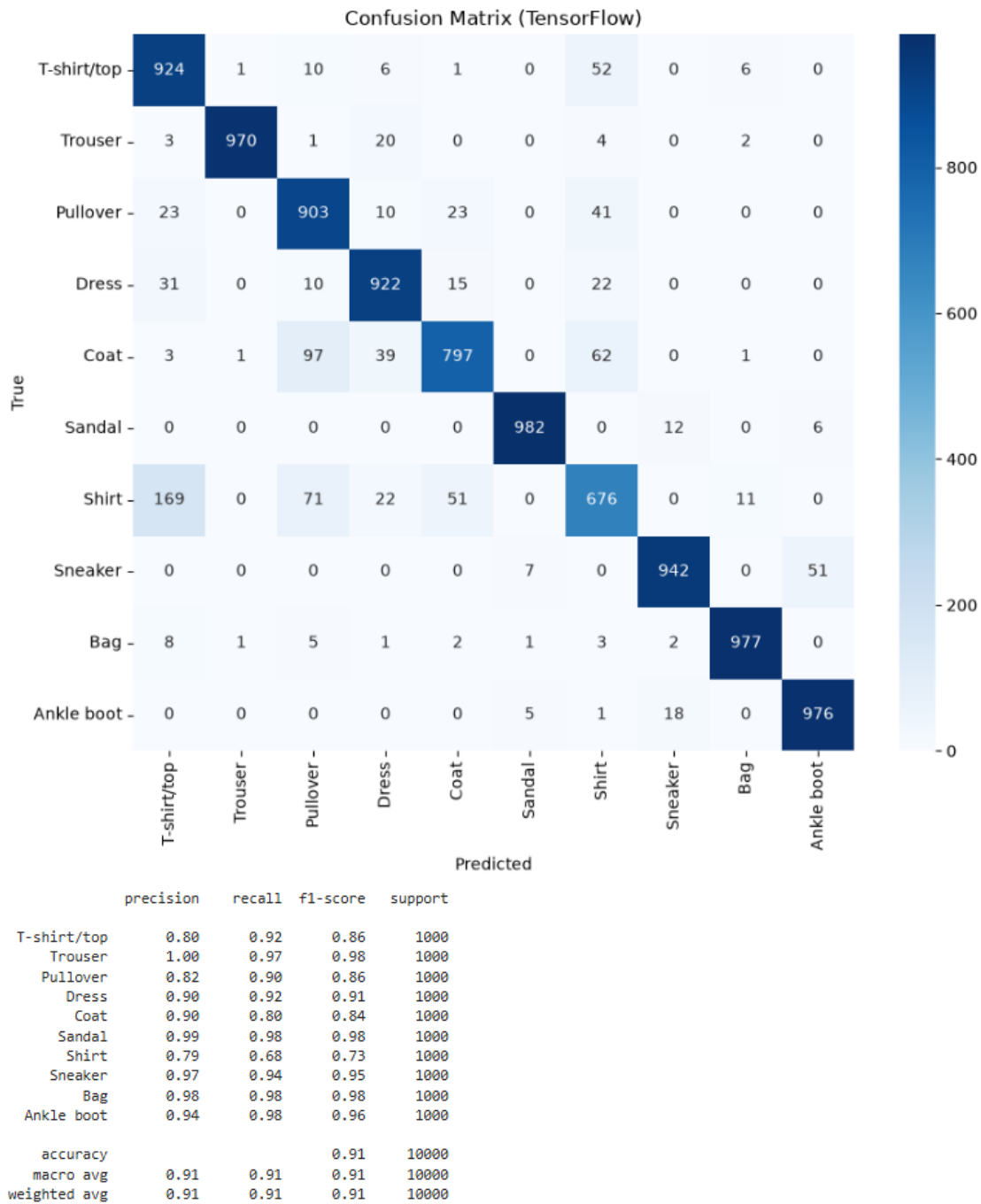
# Dự đoán
y_pred_probs = best_model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (TensorFlow)')
plt.show()

print(classification_report(y_test, y_pred, target_names=class_names))
```

Output:

313/313 ————— 2s 6ms/step



Diễn giải kỹ thuật:

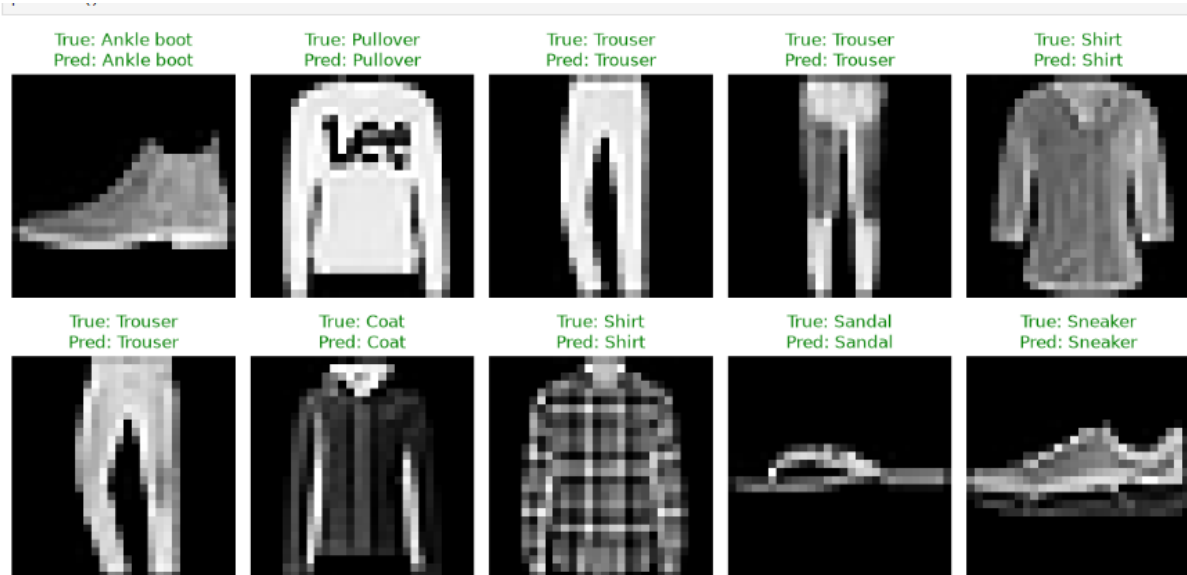
- Khi nạp lại checkpoint .keras tốt nhất, mô hình đạt độ chính xác trung bình (Accuracy) là **0.91** (tương ứng **90.69%**).
- Lớp **Trouser** đạt Precision tuyệt đối \$1.00\$ và F1-score \$0.98\$. Các lớp Sandal, Bag, Ankle boot đều đạt F1-score từ **0.96** đến **0.98**. Lớp **Shirt** đạt F1-score \$0.73\$ (Precision \$0.79\$, Recall \$0.68\$).

4.8. Cell 8: Trục Quan Hóa Dự Đoán Thực Tế trên Tập Kiểm Thử

Code:

```
# Hiển thị một số dự đoán
plt.figure(figsize=(12, 6))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    color = 'green' if y_pred[i] == y_test[i] else 'red'
    plt.title(f"True: {class_names[y_test[i]]}\nPred: {class_names[y_pred[i]]}", color=color)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:



Diễn giải kỹ thuật:

- Trực quan hóa xác nhận hầu hết các mẫu dự đoán đều hiển thị màu xanh lá (dự đoán đúng nhãn trang phục tương ứng), khẳng định năng lực phân loại thực tế của mô hình Keras.

5. PHẦN IV: BẢNG SO SÁNH TỔNG HỢP VÀ ĐÁNH GIÁ ĐA CHIỀU (NUMPY vs. PYTORCH vs. TENSORFLOW)

5.1. Bảng So Sánh Thông Số Kỹ Thuật và Hiệu Năng trên Fashion-MNIST

Tiêu chí Đánh giá	NumPy (From Scratch)	PyTorch (Deep CNN)	TensorFlow / Keras
Quy mô dữ liệu	\$2.000\$ train / \$500\$ test (Subset CPU)	Toàn bộ \$60.000\$ train / \$10.000\$ test	Toàn bộ \$60.000\$ train / \$10.000\$ test
Định dạng Tensor	$\$NCHW\$$ $\$(N, 1, 28, 28)\$$	$\$NCHW\$$ $\$(N, 1, 28, 28)\$$	$\$NHWC\$$ $\$(N, 28, 28, 1)\$$
Số lượng Cell Code	13 Cells	8 Cells	8 Cells

Thiết bị tính toán	CPU thuần	GPU (CUDA)	CPU (oneDNN)
Số lượng Epochs	15 Epochs	10 Epochs	10 Epochs
Batch Size	32	64	64
Bộ tối ưu hóa (Optimizer)	Mini-batch SGD + Momentum	Adam	Adam (
Kỹ thuật chống Overfit	Khởi tạo He Normal	BatchNorm2d + Dropout(0.5)	BatchNormalization + Dropout(0.5)
Tổng số tham số	51.000 tham số	421.642 tham số	225.418 tham số
Độ chính xác Train cuối	95.80%	93%	92.62%\$
Độ chính xác Test (Acc)	83.60% (trên 500 mẫu)	92.11% (trên 10.000 mẫu)	90.69% (trên 10.000 mẫu)
F1-Score Lớp Khó (Shirt)	0.5962	0.76	0.73
F1-Score Lớp Tốt Nhất	0.9905 (Trouser)	0.99 (Trouser, Sandal, Bag)	0.98 (Trouser, Sandal, Bag)
Định dạng Lưu trữ	.npz nén nhị phân	.pth (state_dict)	.keras (Full Model format)